

Warm-Starting **All-Pairs Shortest Paths** with Predictions

Adam Polak

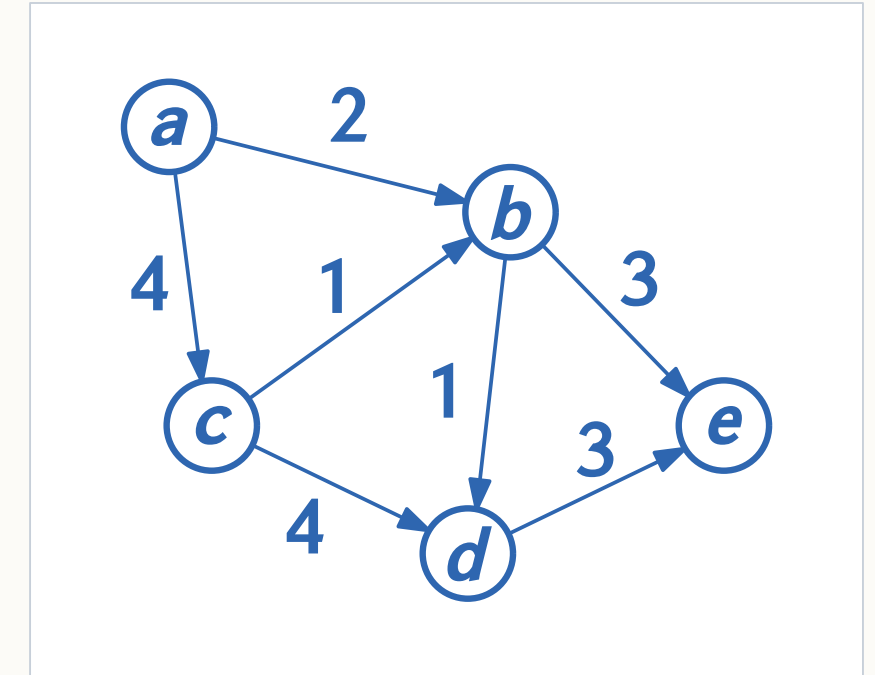
Jonas Schmidt

Bocconi

All-Pairs Shortest Paths (APSP)

All-Pairs Shortest Paths (APSP)

Input: A **directed, weighted** graph $G = (V, E, w)$

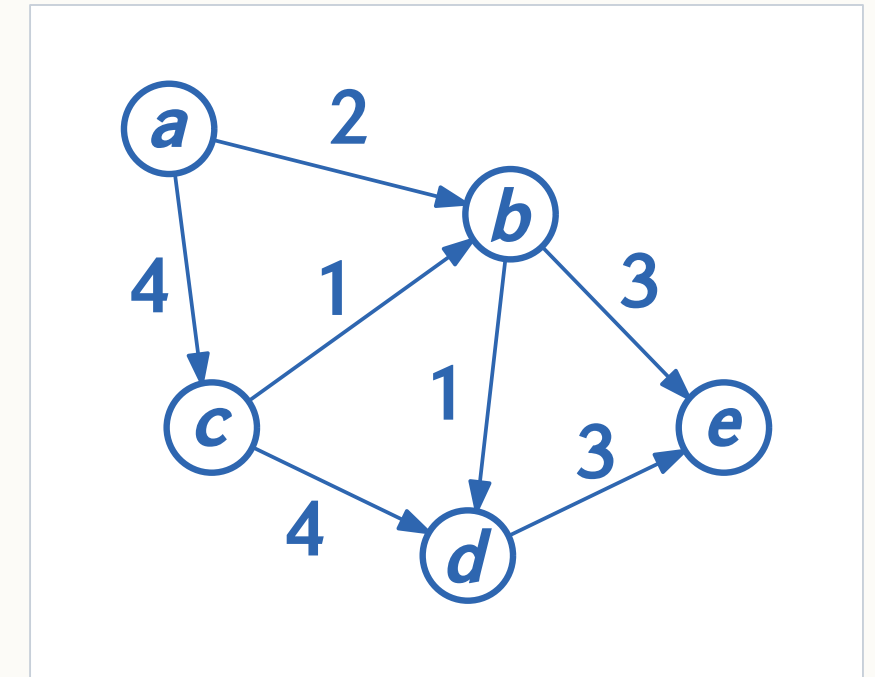


All-Pairs Shortest Paths (APSP)

Input: A **directed, weighted** graph $G = (V, E, w)$

Output: The distance matrix D of G

$$\forall u, v \in V : D[u, v] = \text{dist}_G(u, v)$$



D	a	b	c	d	e
a	0	1	4	3	5
b	∞	0	∞	1	3
c	∞	1	0	2	4
d	∞	∞	∞	0	3
e	∞	∞	∞	∞	0

All-Pairs Shortest Paths (APSP)

Input: A **directed, weighted** graph $G = (V, E, w)$

Output: The distance matrix D of G

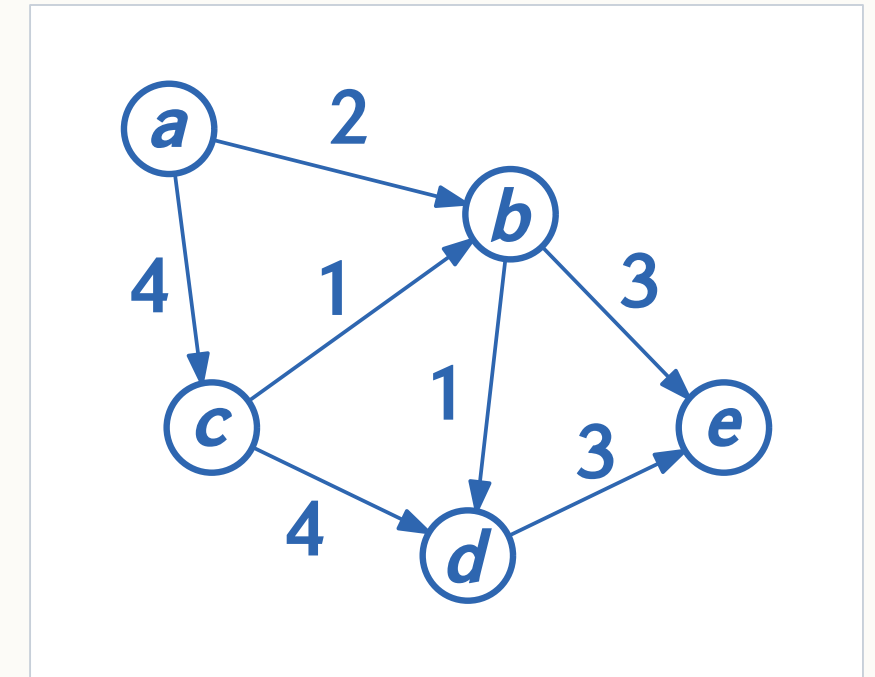
$$\forall u, v \in V : D[u, v] = \text{dist}_G(u, v)$$

Complexity:

► Floyd–Warshall algorithm: $O(n^3)$

[Floyd '62] [Warshall '62]

► $n \times$ SSSP: $\tilde{O}(nm)$



D	a	b	c	d	e
a	0	1	4	3	5
b	∞	0	∞	1	3
c	∞	1	0	2	4
d	∞	∞	∞	0	3
e	∞	∞	∞	∞	0

All-Pairs Shortest Paths (APSP)

Input: A **directed, weighted** graph $G = (V, E, w)$

Output: The distance matrix D of G

$$\forall u, v \in V : D[u, v] = \text{dist}_G(u, v)$$

Complexity:

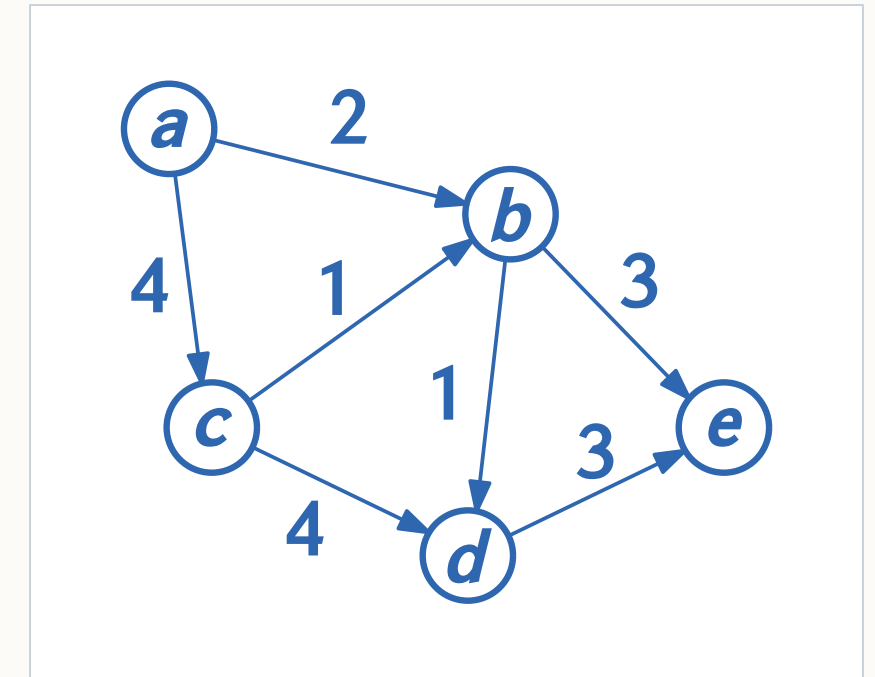
► Floyd–Warshall algorithm: $O(n^3)$

[Floyd '62] [Warshall '62]

► $n \times$ SSSP: $\tilde{O}(nm)$

► State-of-the-art for dense graphs: $\frac{n^3}{2^{\Omega(\sqrt{\log n})}}$

[Williams '14]



D	a	b	c	d	e
a	0	1	4	3	5
b	∞	0	∞	1	3
c	∞	1	0	2	4
d	∞	∞	∞	0	3
e	∞	∞	∞	∞	0

All-Pairs Shortest Paths (APSP)

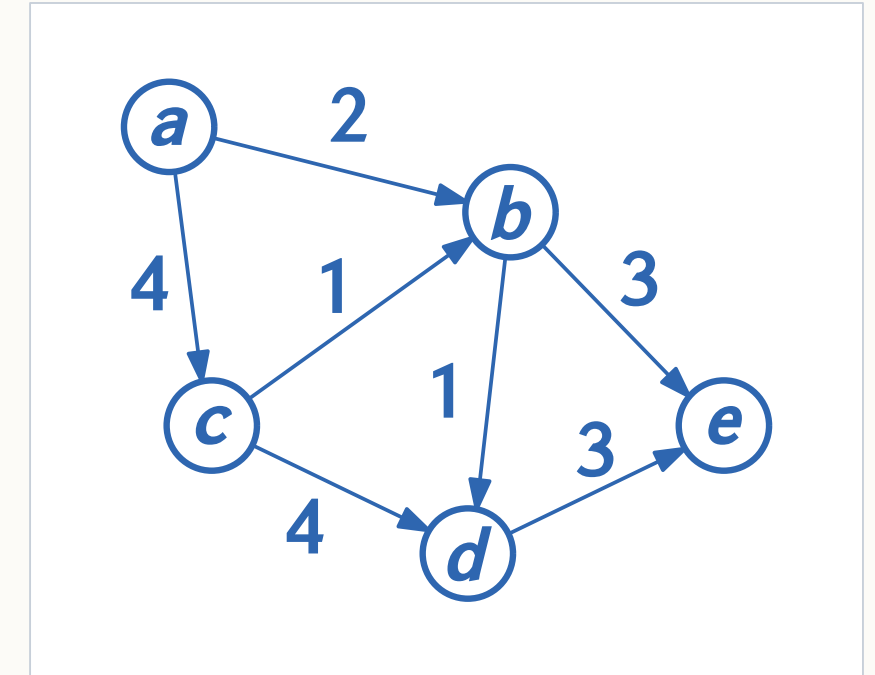
Input: A **directed, weighted** graph $G = (V, E, w)$

Output: The distance matrix D of G

$$\forall u, v \in V : D[u, v] = \text{dist}_G(u, v)$$

Complexity:

- ▶ Floyd–Warshall algorithm: $O(n^3)$ [Floyd '62] [Warshall '62]
- ▶ $n \times$ SSSP: $\tilde{O}(nm)$
- ▶ State-of-the-art for dense graphs: $\frac{n^3}{2^{\Omega(\sqrt{\log n})}}$ [Williams '14]
- ▶ **APSP Hypothesis:** no algorithm in time $O(n^{2.99})$



D	a	b	c	d	e
a	0	1	4	3	5
b	∞	0	∞	1	3
c	∞	1	0	2	4
d	∞	∞	∞	0	3
e	∞	∞	∞	∞	0

All-Pairs Shortest Paths (APSP)

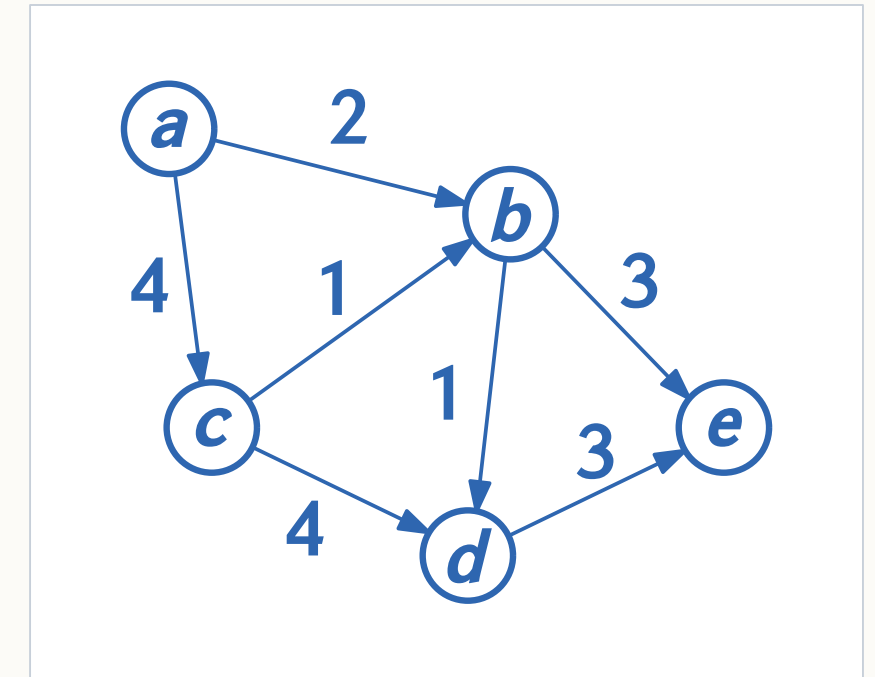
Input: A **directed, weighted** graph $G = (V, E, w)$

Output: The distance matrix D of G

$$\forall u, v \in V : D[u, v] = \text{dist}_G(u, v)$$

Complexity:

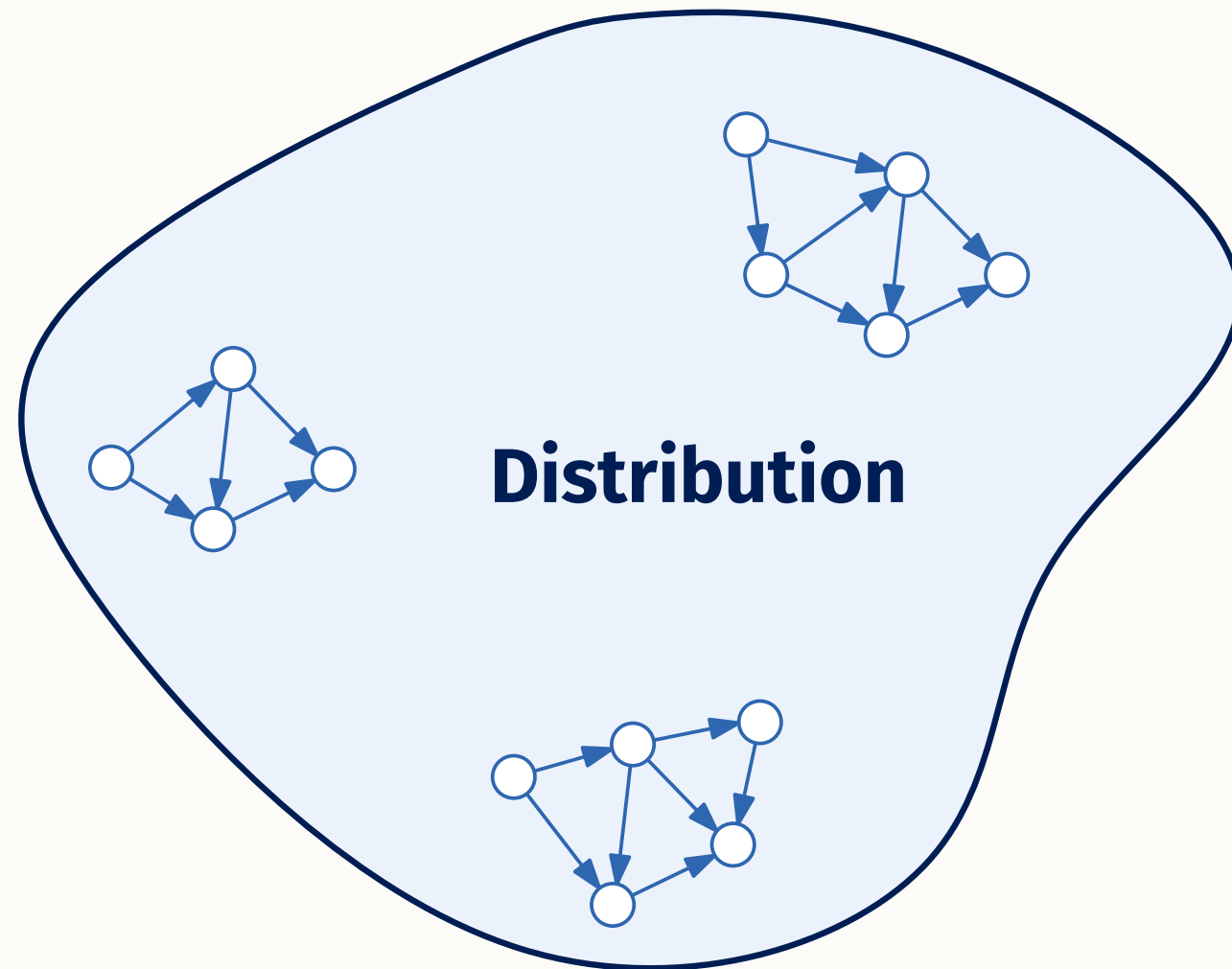
- ▶ Floyd–Warshall algorithm: $O(n^3)$ [Floyd '62] [Warshall '62]
- ▶ $n \times$ SSSP: $\tilde{O}(nm)$
- ▶ State-of-the-art for dense graphs: $\frac{n^3}{2^{\Omega(\sqrt{\log n})}}$ [Williams '14]
- ▶ **APSP Hypothesis:** no algorithm in time $O(n^{2.99})$



D	a	b	c	d	e
a	0	1	4	3	5
b	∞	0	∞	1	3
c	∞	1	0	2	4
d	∞	∞	∞	0	3
e	∞	∞	∞	∞	0

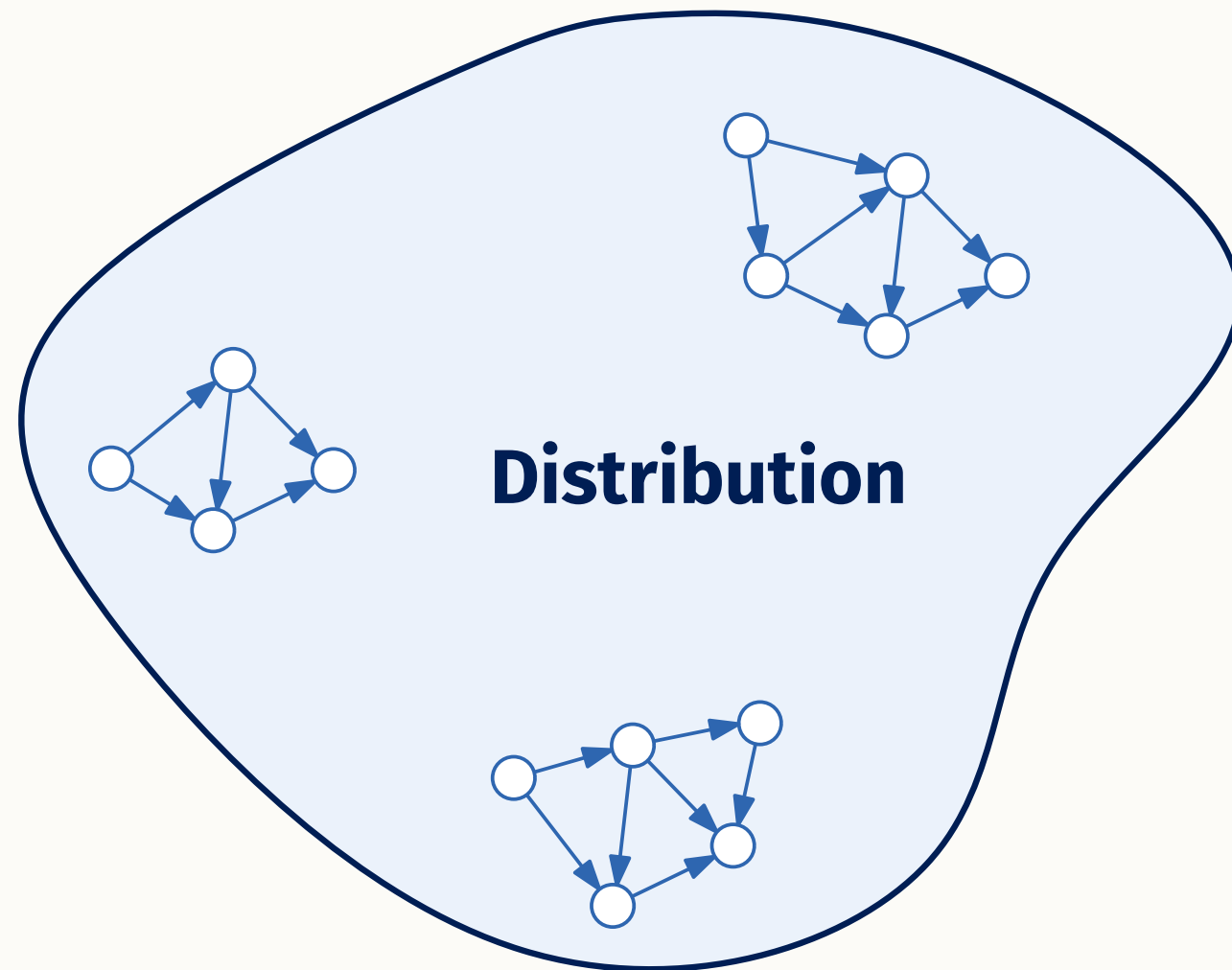
Can we **use predictions** to go below the APSP Hypothesis?

Warm-Starting APSP



Q: Can we use **common structure** to solve instances faster (than n^3)?

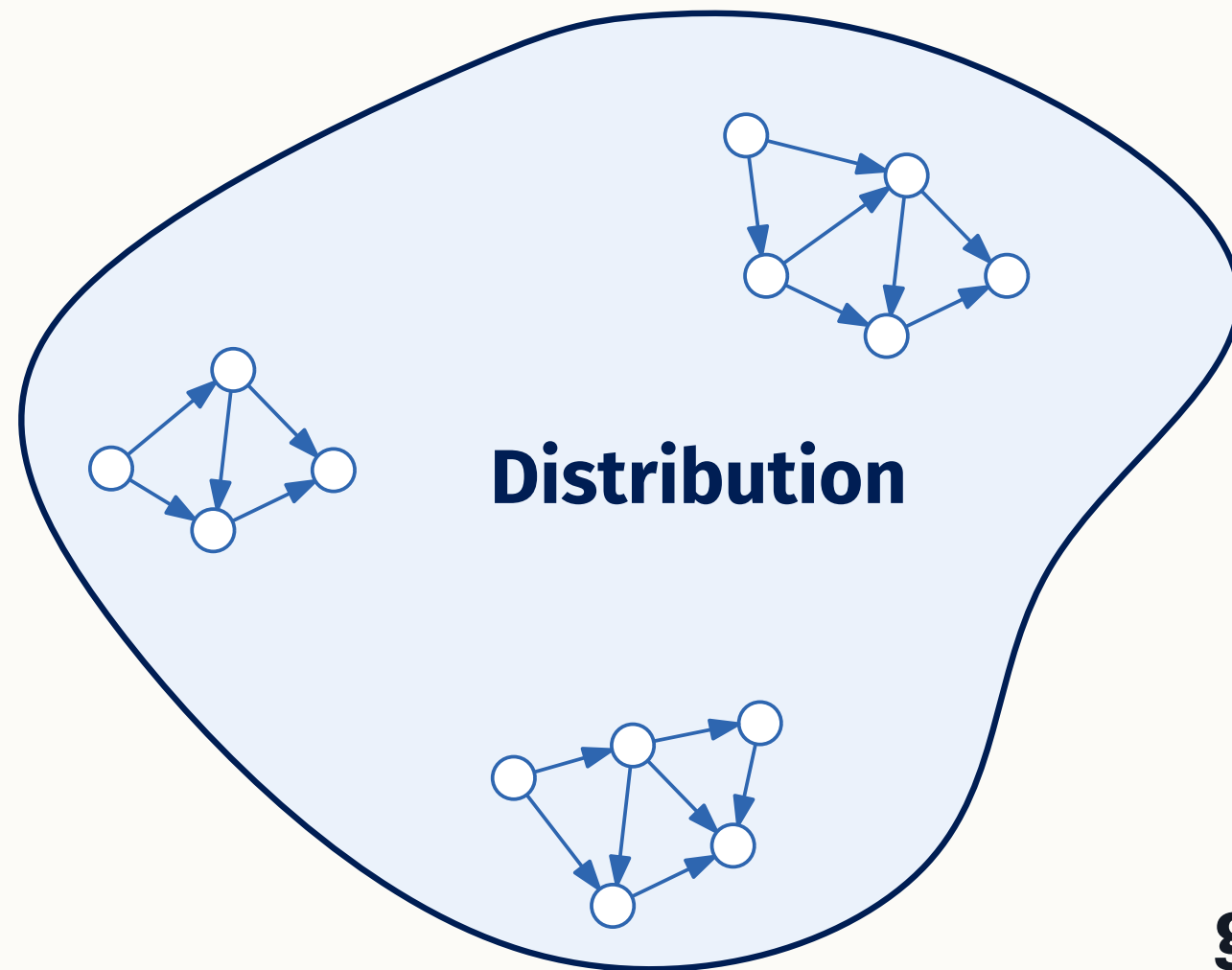
Warm-Starting APSP



Q: Can we use **common structure** to solve instances faster (than n^3)?

Algs with Predictions: Learn some **prediction** to speed up the algorithm!

Warm-Starting APSP



Q: Can we use **common structure** to solve instances faster (than n^3)?

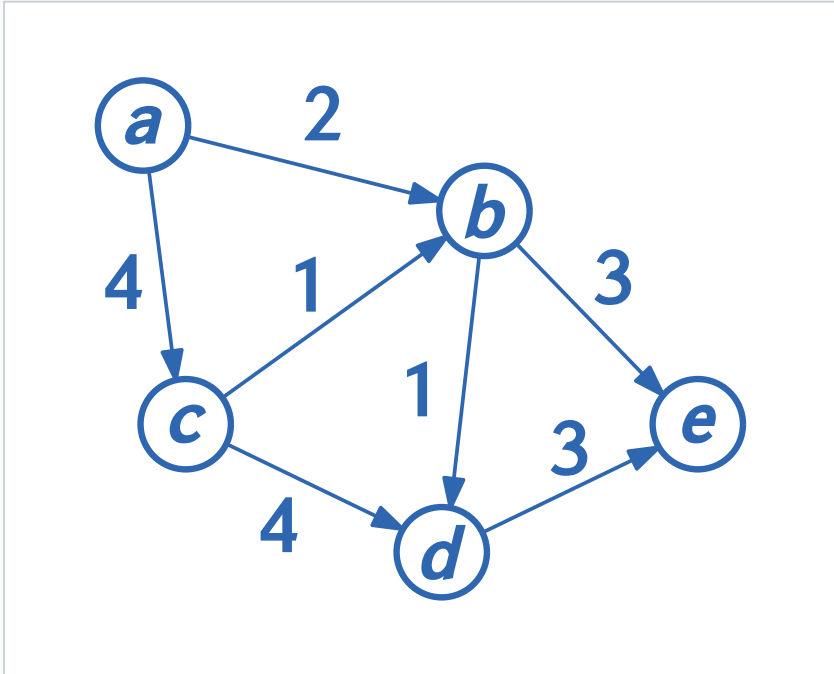
Algs with Predictions: Learn some **prediction** to speed up the algorithm!

good prediction:
provable fast alg

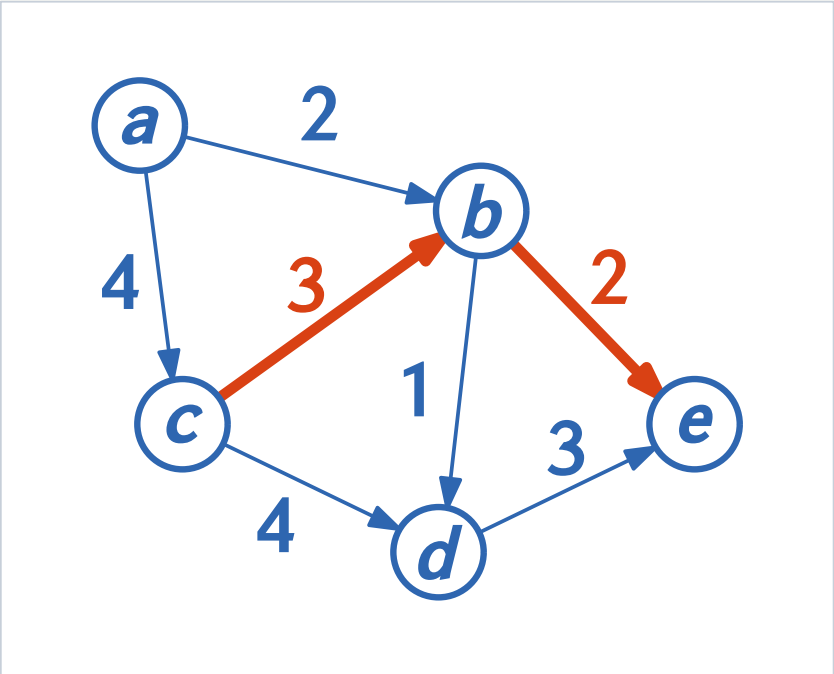
bad prediction:
worst-case alg

What kind of structure?

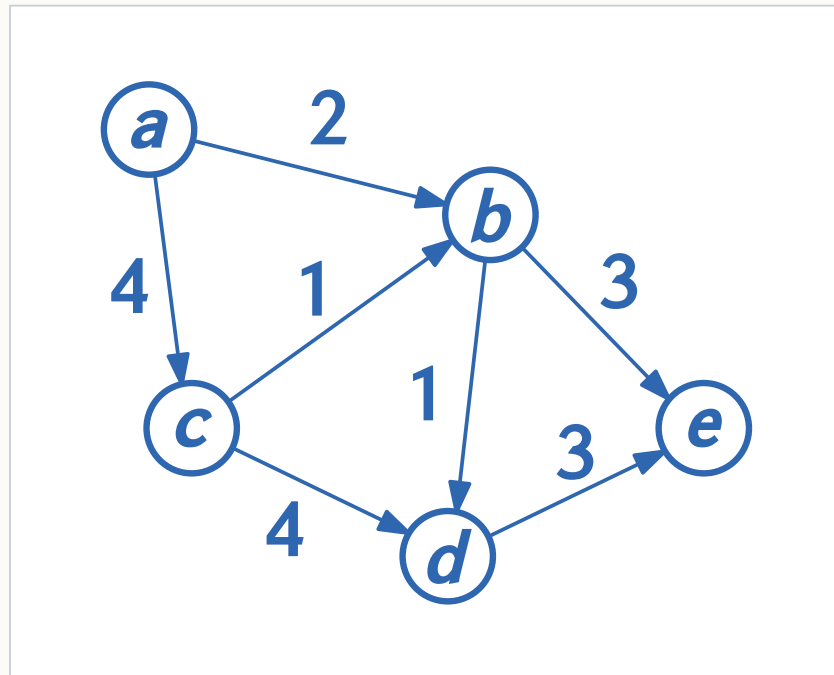
What kind of structure?



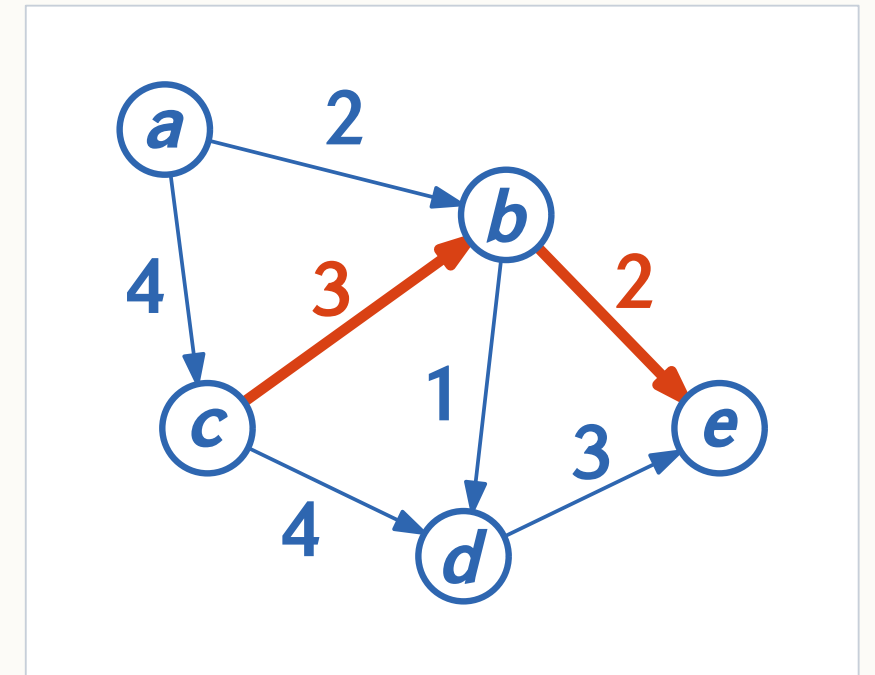
structure = **few input changes??**



What kind of structure?



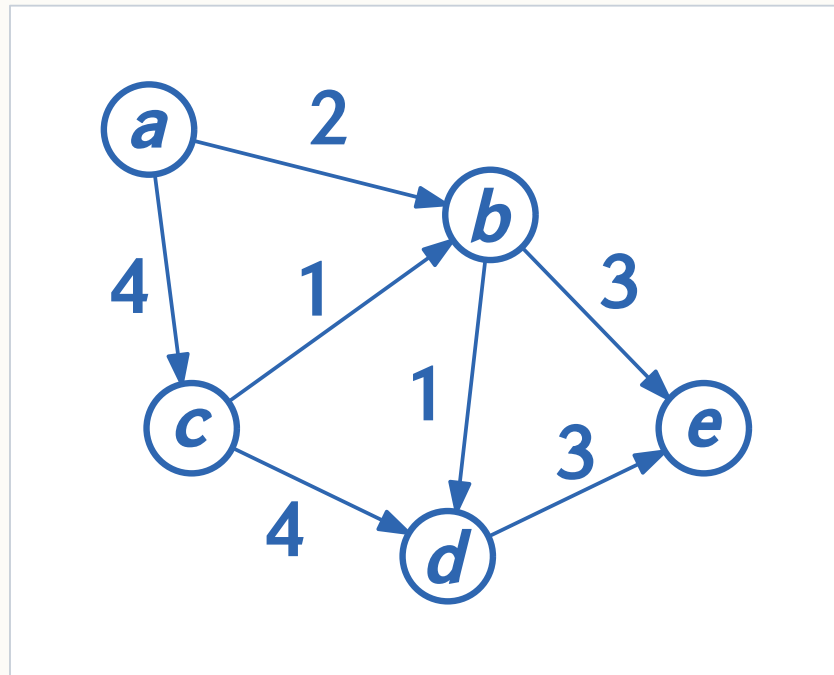
structure = ~~few input changes??~~



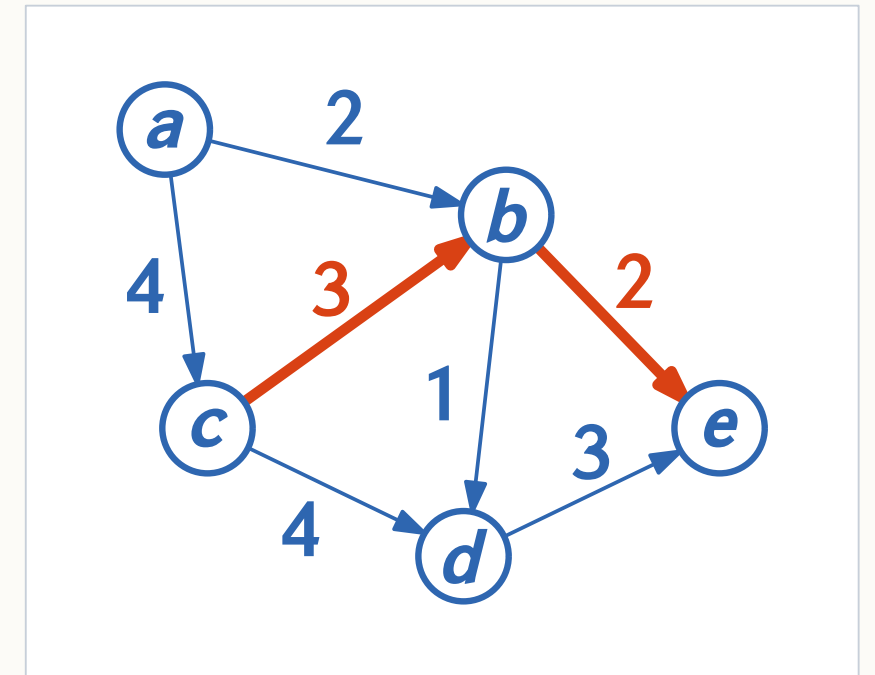
Use **dynamic algorithms** instead!

- Subcubic algorithm exists for less than $\sqrt{n}$ vertices changed [Mao '24]

What kind of structure?



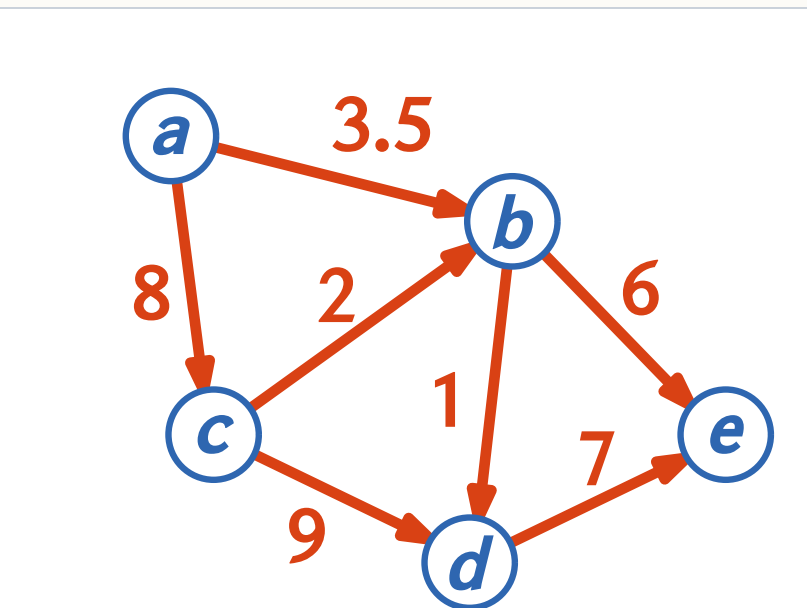
structure = **few input changes??**



Use **dynamic algorithms** instead!

- Subcubic algorithm exists for less than $\sqrt{n}$ vertices changed [Mao '24]

different numbers, similar **path structure**
e.g., road network during rush hours



multiply all edge weights by ≈ 2

APSP with Predictions: Approach 1

APSP with Predictions: Approach 1

What we would like to have:

APSP instance $G = (V, E, w)$

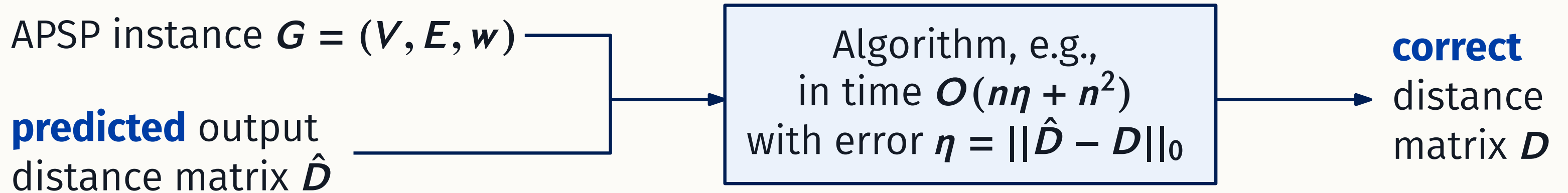
predicted output
distance matrix $\hat{D}$

Algorithm, e.g.,
in time $O(n\eta + n^2)$
with error $\eta = ||\hat{D} - D||_0$

correct
distance
matrix D

APSP with Predictions: Approach 1

What we would like to have:



Issue:

Theorem

[Vassilevska Williams, Williams '10]

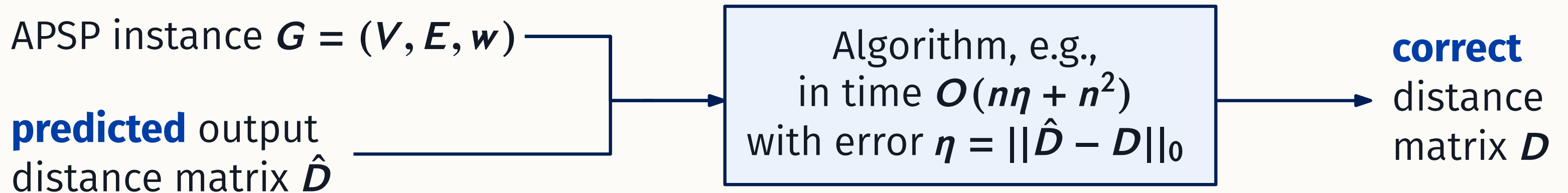
If there is a $O(n^{2.99})$ -time algorithm that **verifies** if a given matrix $\hat{D}$ is the correct distance matrix, the APSP hypothesis is false.

APSP

APSP Verification

APSP with Predictions: Approach 1

What we would like to have:



Issue:

- ▶ Verifying $\hat{D}$ is as hard as computing D

Theorem

[Vassilevska Williams, Williams '10]

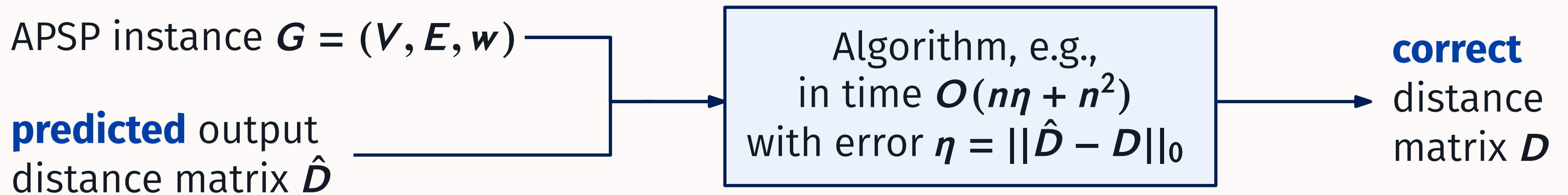
If there is a $O(n^{2.99})$ -time algorithm that **verifies** if a given matrix $\hat{D}$ is the correct distance matrix, the APSP hypothesis is false.

APSP

APSP Verification

APSP with Predictions: Approach 1

What we would like to have:



Issue:

- ▶ Verifying $\hat{D}$ is as hard as computing D
- ▶ Approach **cannot work** with **any reasonable η** !

Theorem

[Vassilevska Williams, Williams '10]

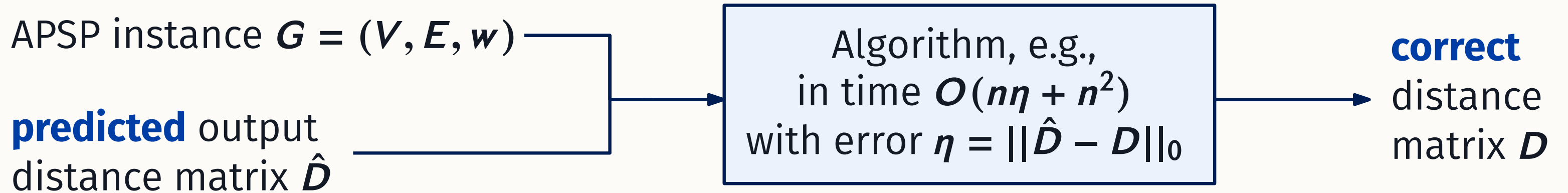
If there is a $O(n^{2.99})$ -time algorithm that **verifies** if a given matrix $\hat{D}$ is the correct distance matrix, the APSP hypothesis is false.

APSP

APSP Verification

APSP with Predictions: Approach 1

What we would like to have:



Issue:

- ▶ Verifying $\hat{D}$ is as hard as computing D
- ▶ Approach **cannot work** with **any reasonable η** !

Theorem

[Vassilevska Williams, Williams '10]

If there is a $O(n^{2.99})$ -time algorithm that **verifies** if a given matrix $\hat{D}$ is the correct distance matrix, the APSP hypothesis is false.

We need a **stronger prediction!**

APSP

APSP Verification

(Co-)nondeterministic APSP

(Co-)nondeterministic APSP

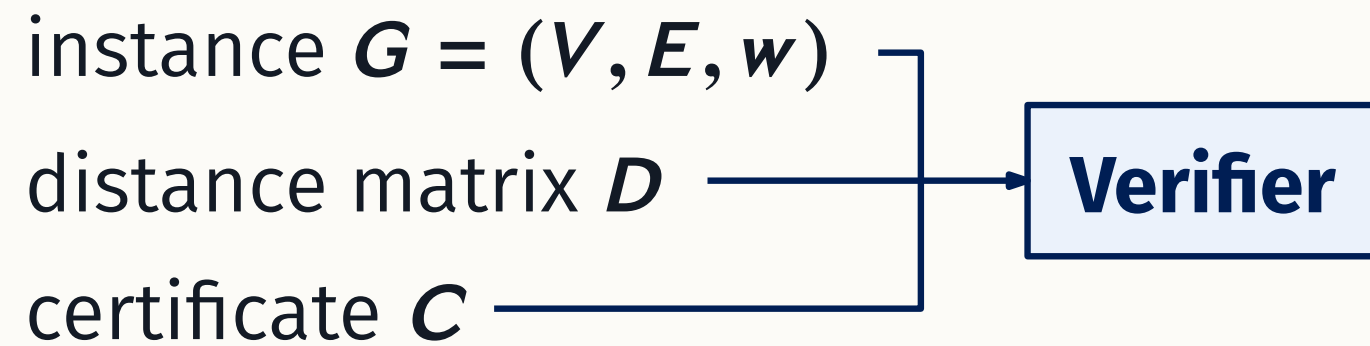
- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?

(Co-)nondeterministic APSP

- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?
- ▶ Yes! Co-nondeterministic algorithms for APSP

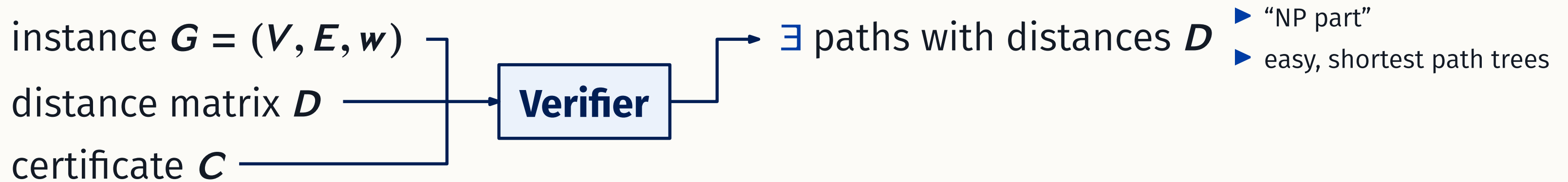
(Co-)nondeterministic APSP

- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?
- ▶ Yes! Co-nondeterministic algorithms for APSP



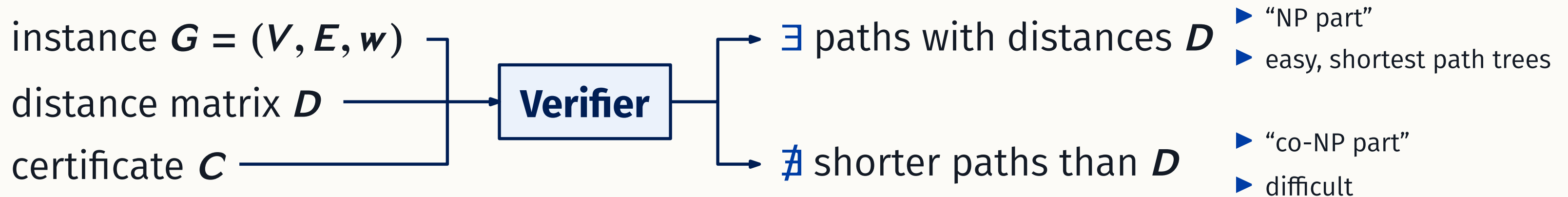
(Co-)nondeterministic APSP

- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?
- ▶ Yes! Co-nondeterministic algorithms for APSP



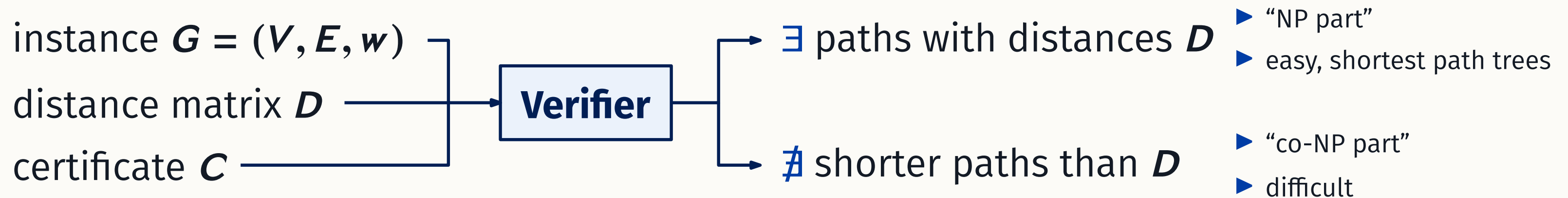
(Co-)nondeterministic APSP

- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?
- ▶ Yes! Co-nondeterministic algorithms for APSP



(Co-)nondeterministic APSP

- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?
- ▶ Yes! Co-nondeterministic algorithms for APSP

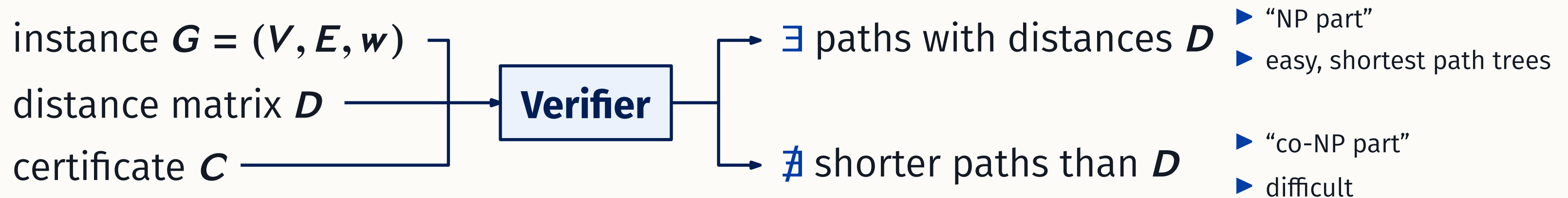


2 co-nondeterministic algorithms known:

- ▶ Hashing-based: time $O(n^{(\omega+3)/2})$ [Carmosino, Gao, Impagliazzo, Mihajlin, Paturi, Schneider '17]
- ▶ More structural / real inputs: time $O(n^{(\omega+6)/3})$ [Chan, Vassilevska Williams, Xu '23]

(Co-)nondeterministic APSP

- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?
- ▶ Yes! Co-nondeterministic algorithms for APSP

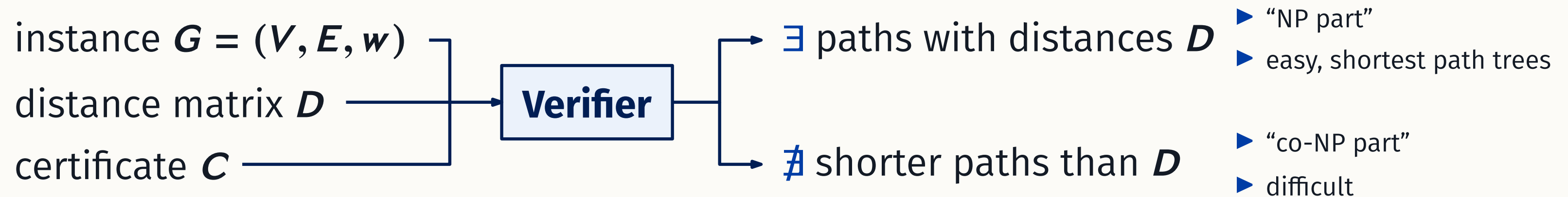


2 co-nondeterministic algorithms known:

- ▶ Hashing-based: time $O(n^{2.5})$ [Carmosino, Gao, Impagliazzo, Mihajlin, Paturi, Schneider '17]
- ▶ More structural / real inputs: time $O(n^{2.66})$ [Chan, Vassilevska Williams, Xu '23]

(Co-)nondeterministic APSP

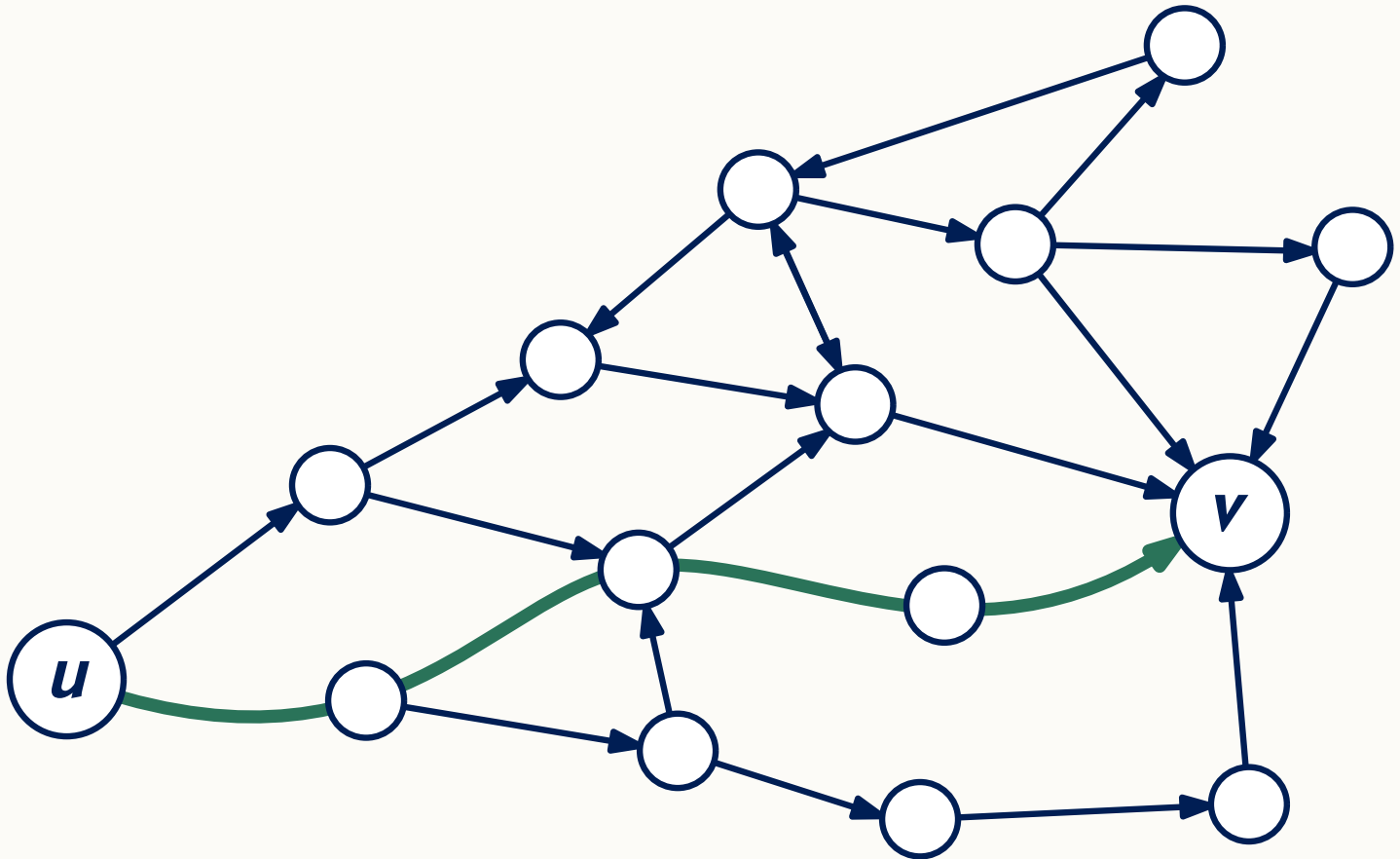
- ▶ Does there even exist a prediction with a subcubic algorithm if $\eta = 0$ ($\hat{D} = D$)?
- ▶ Yes! Co-nondeterministic algorithms for APSP



2 co-nondeterministic algorithms known:

- ▶ Hashing-based: time $O(n^{2.5})$ [Carmosino, Gao, Impagliazzo, Mihajlin, Paturi, Schneider '17]
- ▶ More structural / real inputs: time $O(n^{2.66})$ [Chan, Vassilevska Williams, Xu '23]

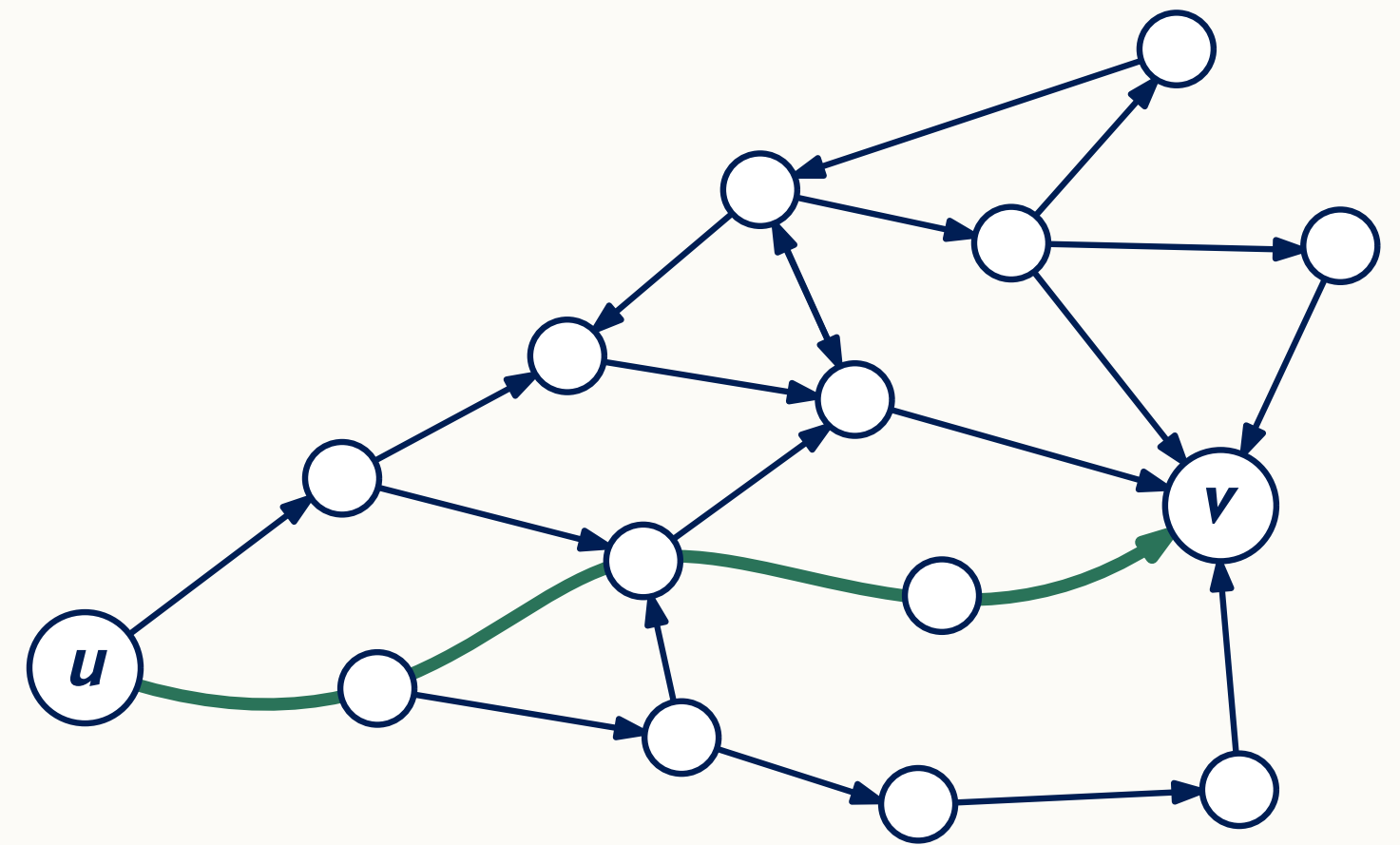
Our Result (Informal)



Our Result (Informal)

Certificate / Prediction C : (informal)

For every vertex pair (u, v) , vertices **causing the shortest detour** on a u - v -path.

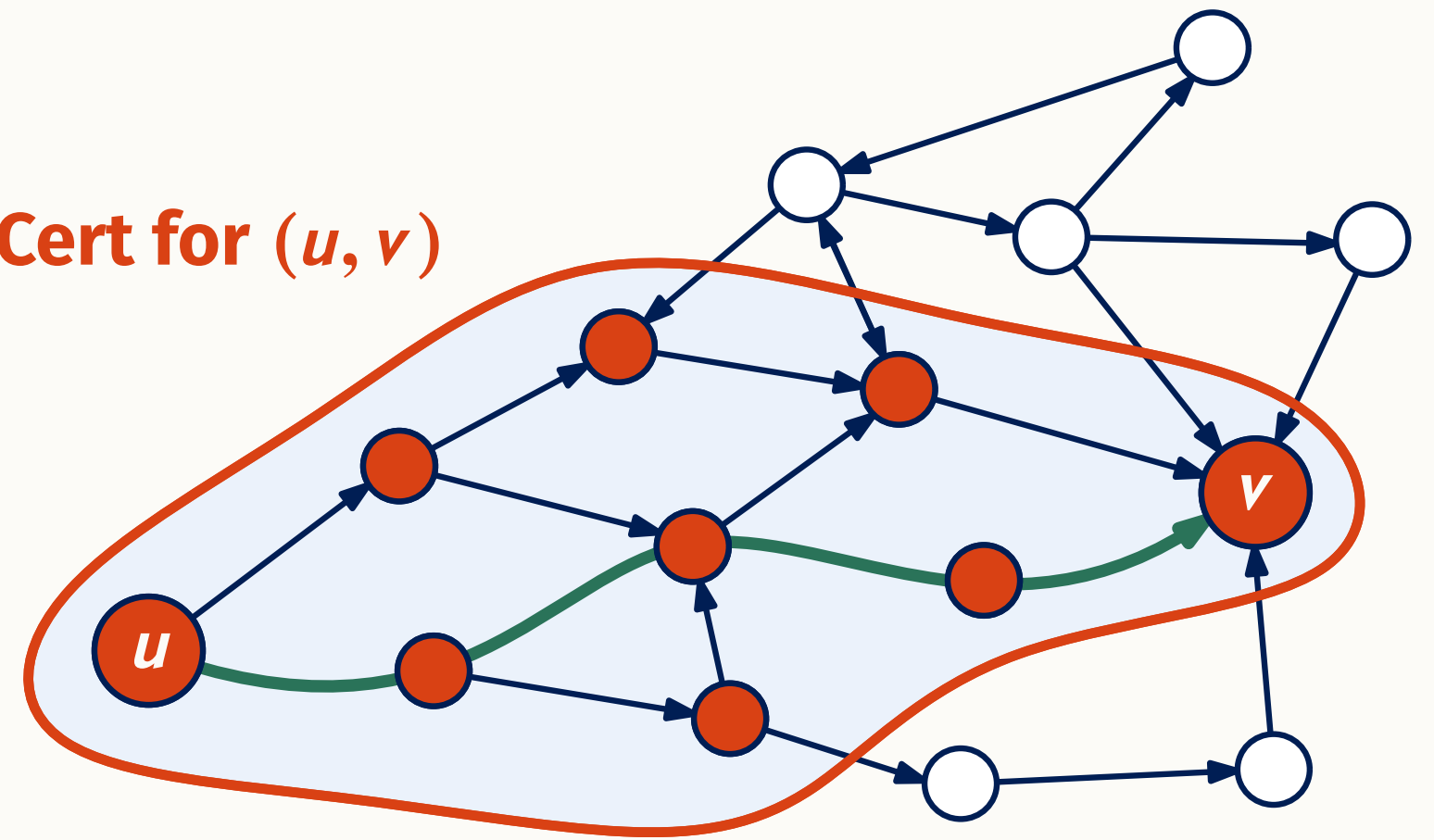


Our Result (Informal)

Certificate / Prediction C : (informal)

For every vertex pair (u, v) , vertices **causing the shortest detour** on a u - v -path.

Cert for (u, v)

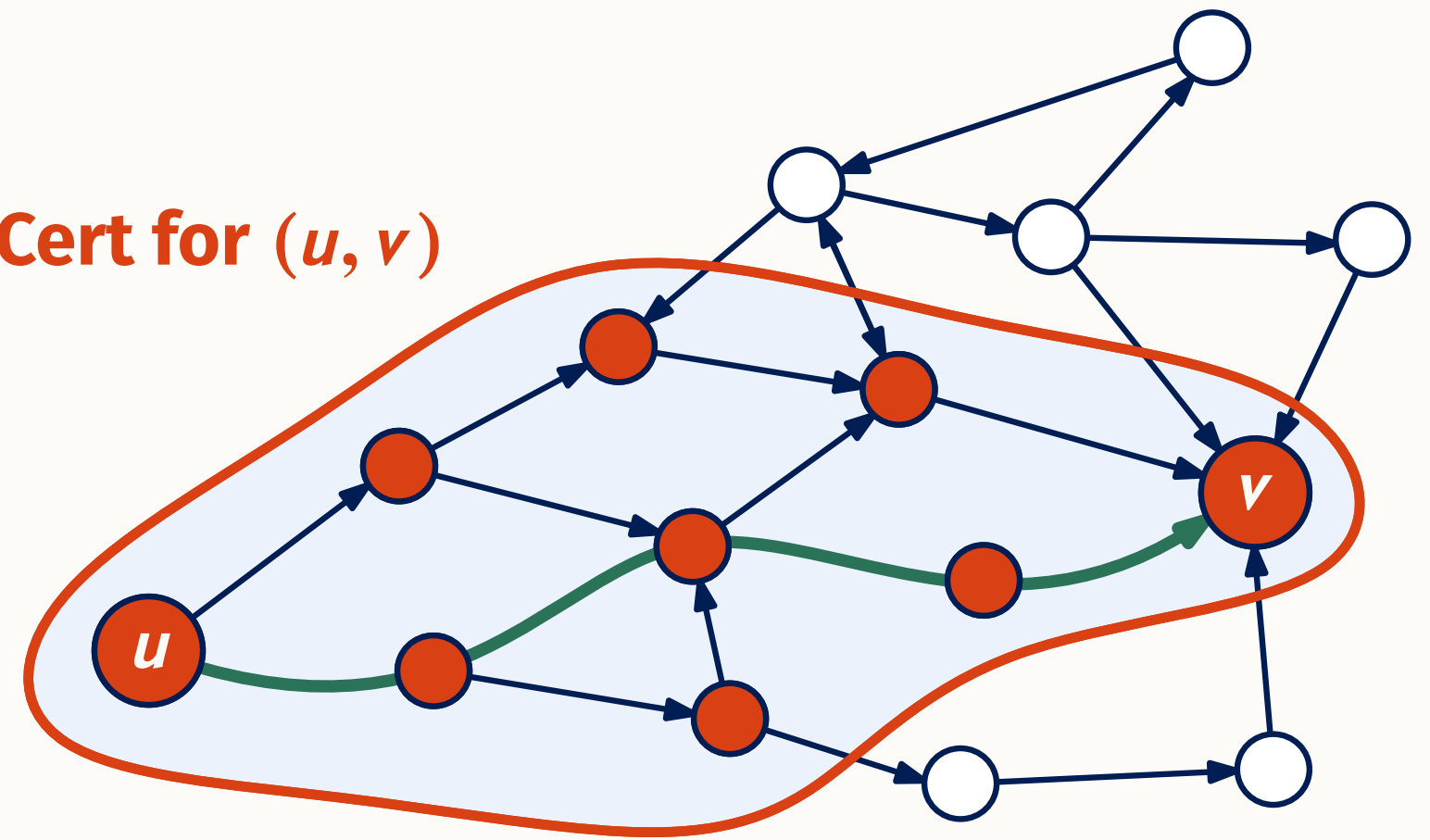


Our Result (Informal)

Certificate / Prediction $\mathcal{C}$: (informal)

For every vertex pair (u, v) , vertices **causing the shortest detour** on a u - v -path.

Cert for (u, v)



Error Measure η : (informal)

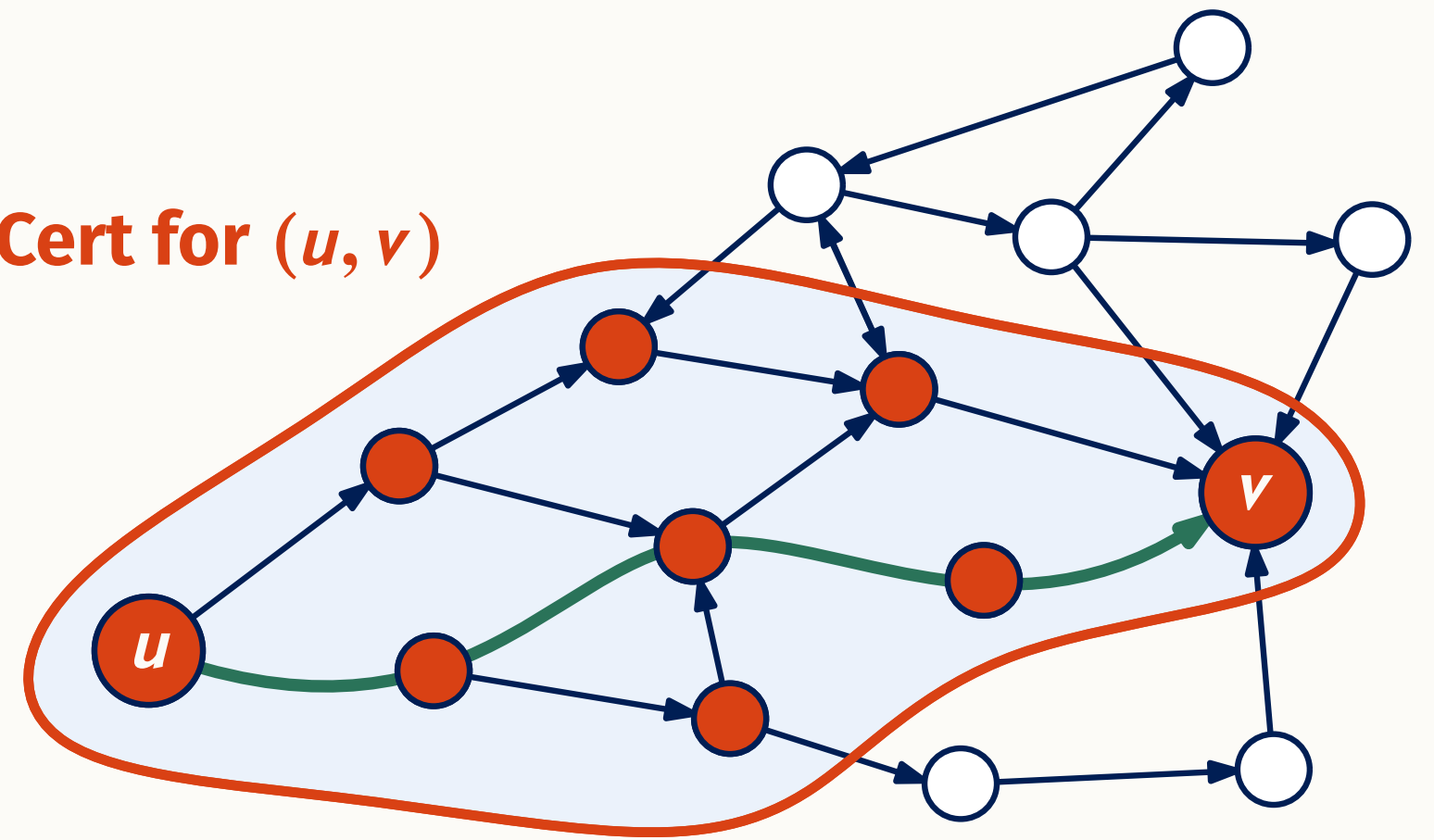
The number of (u, v) where a **very short detour is missing** or **cannot deduce $\text{dist}(u, v)$** from $\mathcal{C}$.

Our Result (Informal)

Certificate / Prediction C : (informal)

For every vertex pair (u, v) , vertices **causing the shortest detour** on a u - v -path.

Cert for (u, v)



Error Measure η : (informal)

The number of (u, v) where a **very short detour is missing** or **cannot deduce $\text{dist}(u, v)$** from C .

Theorem

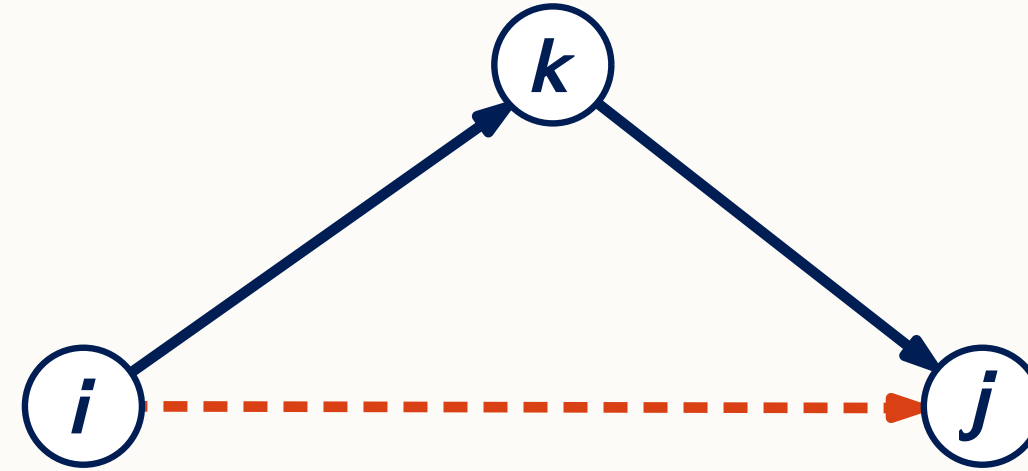
Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

The Verifier

Reminder: Relaxations

Reminder: Relaxations

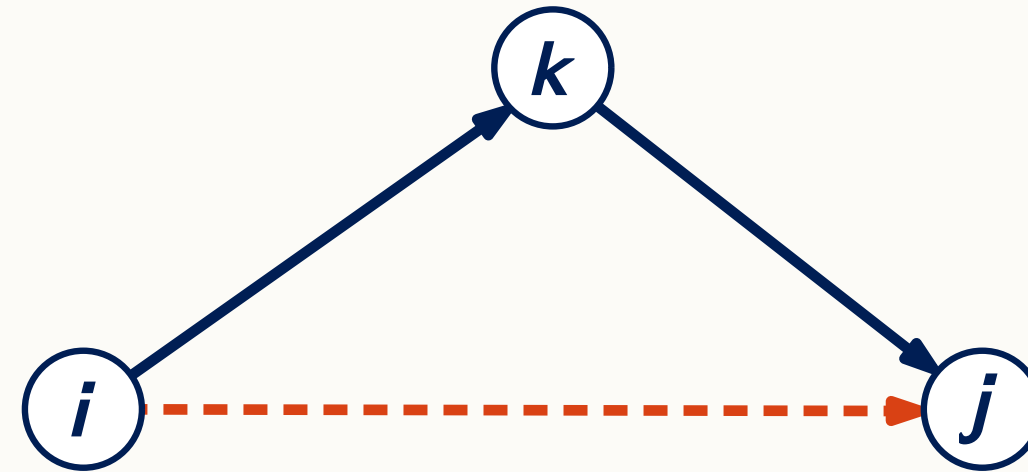
Relaxation of (i, j) with k :



$$D[i, j] = \min \{ D[i, j], D[i, k] + D[k, j] \}$$

Reminder: Relaxations

Relaxation of (i, j) with k :

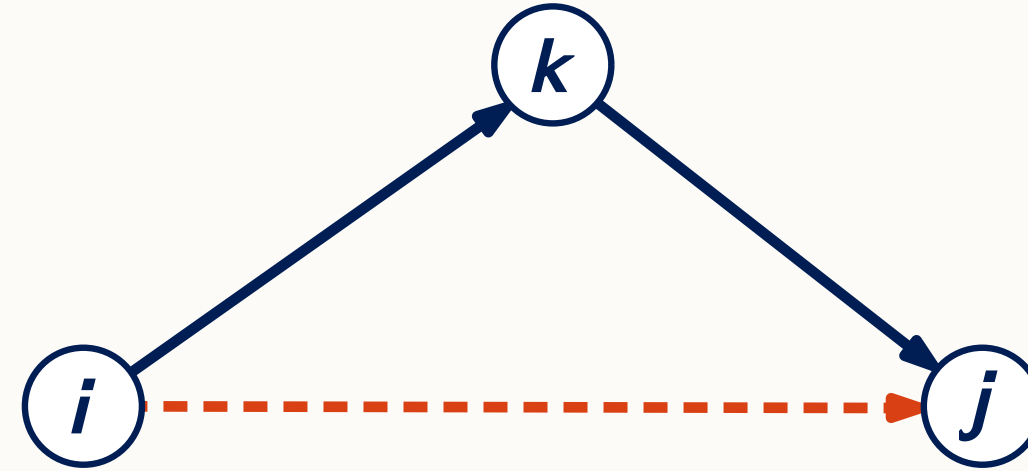


$$D[i, j] = \min \{ D[i, j], D[i, k] + D[k, j] \}$$

- ▶ n^3 possible relaxations

Reminder: Relaxations

Relaxation of (i, j) with k :

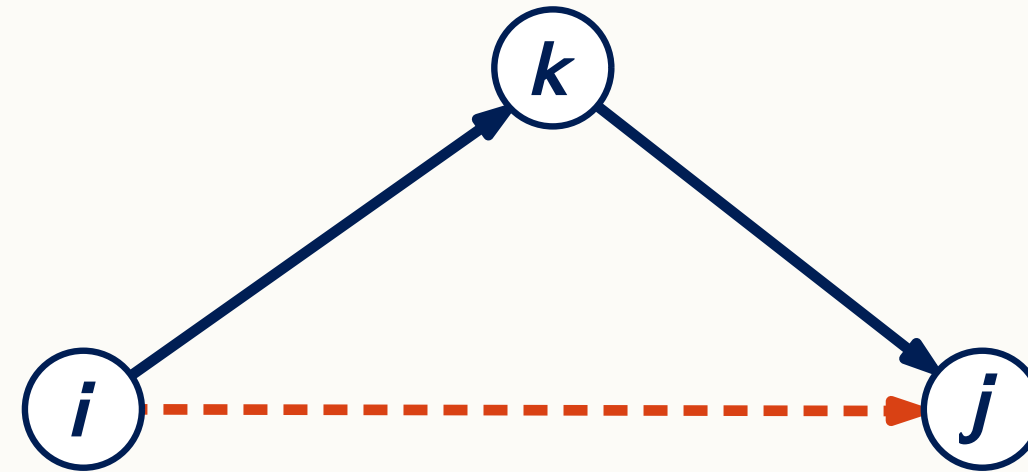


$$D[i, j] = \min \{ D[i, j], D[i, k] + D[k, j] \}$$

- ▶ n^3 possible relaxations
- ▶ D contains correct distances $\iff$ $\nexists$ decreasing relaxation
(Assuming D contains only path lengths $\leq$ direct edge weight)

Reminder: Relaxations

Relaxation of (i, j) with k :

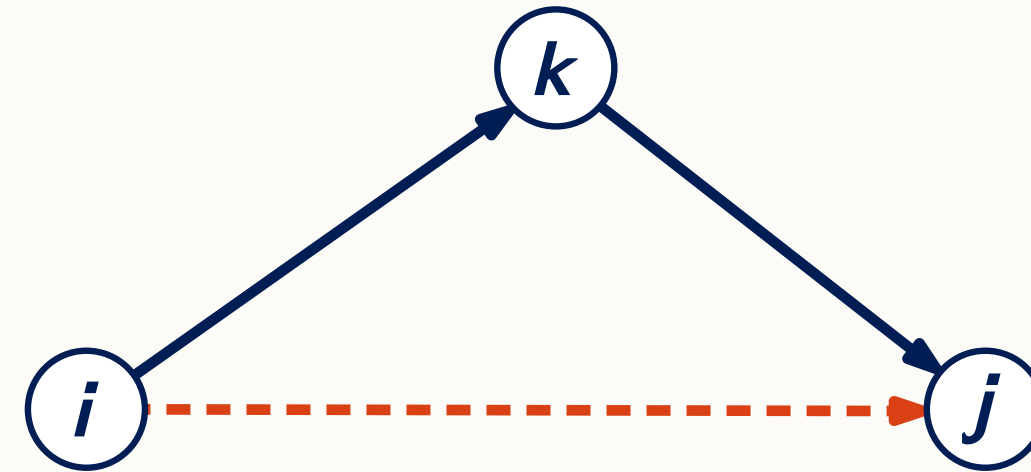


$$D[i, j] = \min \{ D[i, j], D[i, k] + D[k, j] \}$$

- ▶ n^3 possible relaxations
 - ▶ D contains correct distances $\iff$ $\nexists$ decreasing relaxation
 - ▶ A = adjacency matrix (with edge weights)
- (Assuming D contains only path lengths $\leq$ direct edge weight)

Reminder: Relaxations

Relaxation of (i, j) with k :



$$D[i, j] = \min \{ D[i, j], D[i, k] + D[k, j] \}$$

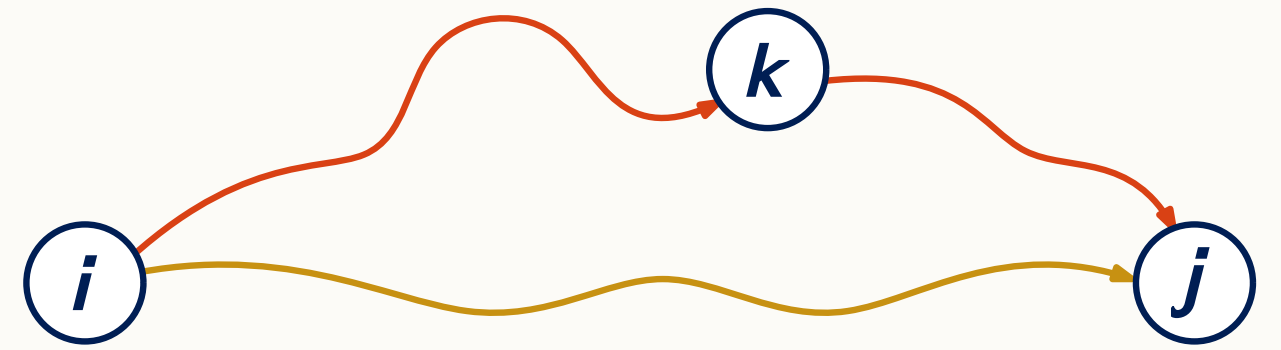
- ▶ n^3 possible relaxations
- ▶ D contains correct distances $\iff$ $\nexists$ decreasing relaxation
- ▶ A = adjacency matrix (with edge weights) (Assuming D contains only path lengths $\leq$ direct edge weight)

Theorem (Floyd–Warshall) [Floyd '62] [Warshall '62]

$\exists$ permutation of all n^3 relaxations s.t., after performing them once to A , it is the correct distance matrix.

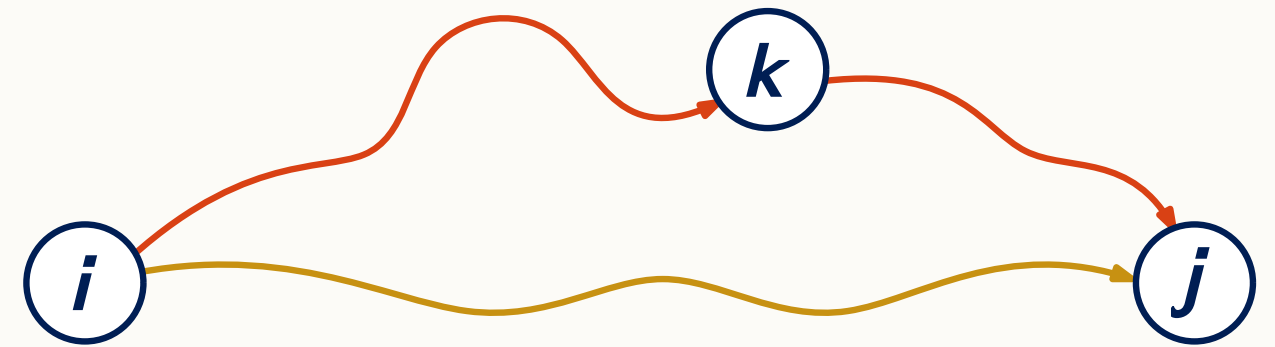
Sketch: **APSP Verifier [CVWX]**

Sketch: APSP Verifier [CVWX]



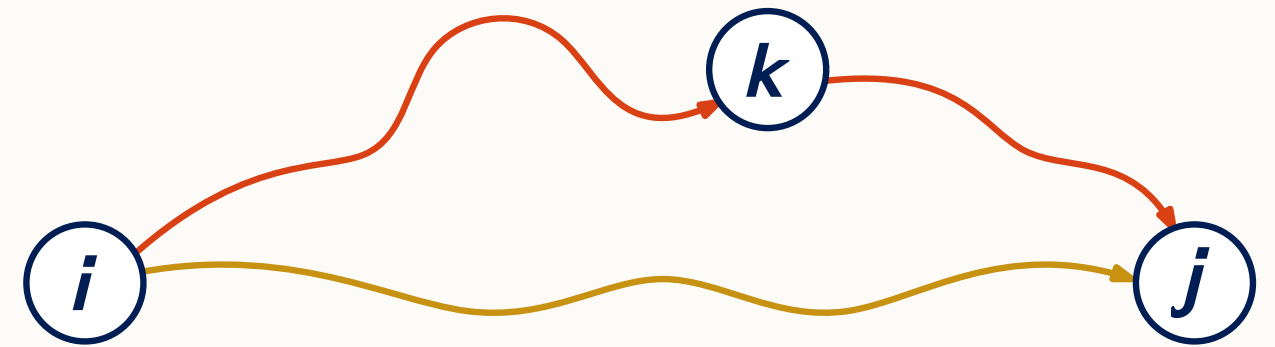
- Define the **detour of i and j via k** as $\text{detour}_{i,j}(k) := D[i, k] + D[k, j] - D[i, j]$

Sketch: APSP Verifier [CVWX]



- ▶ Define the **detour of i and j via k** as $\text{detour}_{i,j}(k) := D[i, k] + D[k, j] - D[i, j]$
- ▶ **Intuition:** Small detour $\implies k$ close to shortest path, good relaxation

Sketch: APSP Verifier [CVWX]

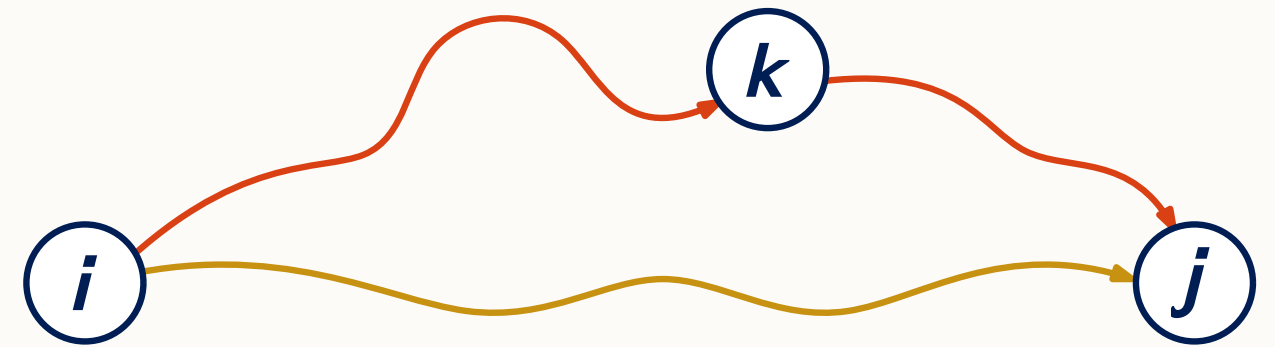


- ▶ Define the **detour of i and j via k** as $\text{detour}_{i,j}(k) := D[i, k] + D[k, j] - D[i, j]$
- ▶ **Intuition:** Small detour $\implies k$ close to shortest path, good relaxation

Certificate:

A matrix C where $C[i, j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

Sketch: APSP Verifier [CVWX]



- ▶ Define the **detour of i and j via k** as $\text{detour}_{i,j}(k) := D[i, k] + D[k, j] - D[i, j]$
- ▶ **Intuition:** Small detour $\implies k$ close to shortest path, good relaxation

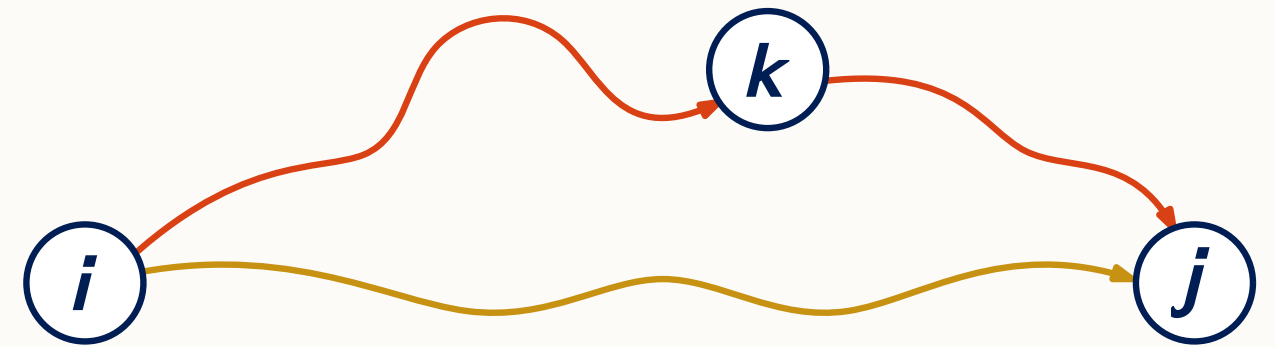
Certificate:

A matrix $\mathcal{C}$ where $\mathcal{C}[i, j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

Claim:

Can verify distance matrix D in time $\tilde{O}(n^{2.75})$, given correct certificate $\mathcal{C}$.

Sketch: APSP Verifier [CVWX]



- ▶ Define the **detour of i and j via k** as $\text{detour}_{i,j}(k) := D[i, k] + D[k, j] - D[i, j]$
- ▶ **Intuition:** Small detour $\implies k$ close to shortest path, good relaxation

Certificate:

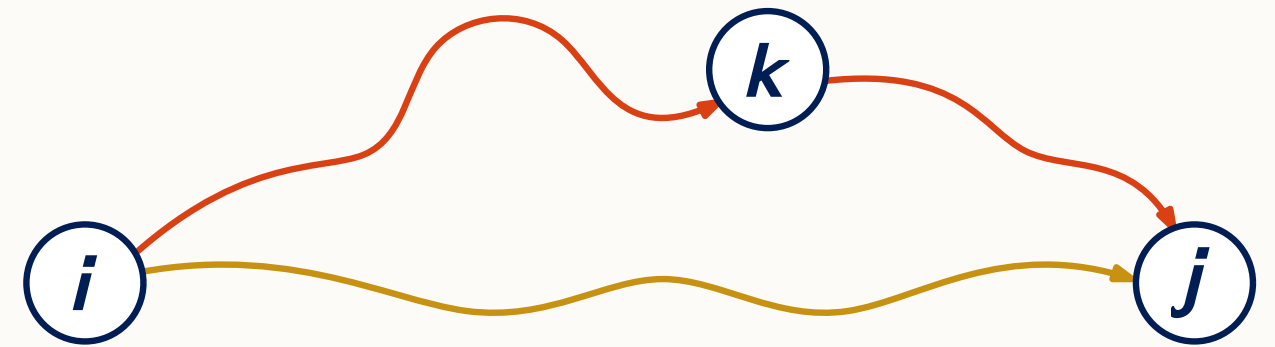
A matrix $\mathcal{C}$ where $\mathcal{C}[i, j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

Claim:

Can verify distance matrix D in time $\tilde{O}(n^{2.75})$, given correct certificate $\mathcal{C}$.



Sketch: APSP Verifier [CVWX]



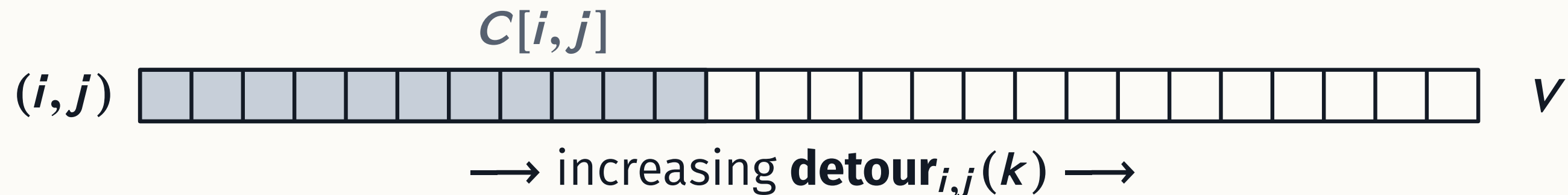
- ▶ Define the **detour of i and j via k** as $\text{detour}_{i,j}(k) := D[i, k] + D[k, j] - D[i, j]$
- ▶ **Intuition:** Small detour $\implies k$ close to shortest path, good relaxation

Certificate:

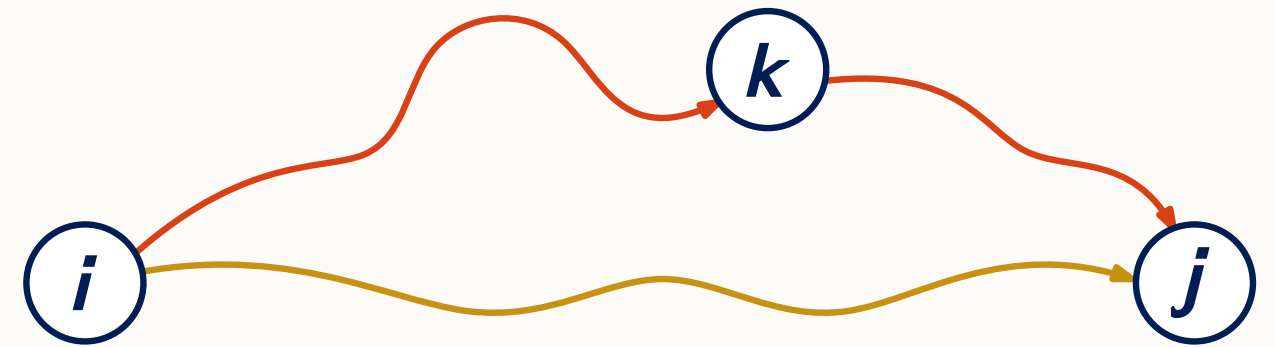
A matrix $\mathcal{C}$ where $\mathcal{C}[i, j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

Claim:

Can verify distance matrix D in time $\tilde{O}(n^{2.75})$, given correct certificate $\mathcal{C}$.



Sketch: APSP Verifier [CVWX]



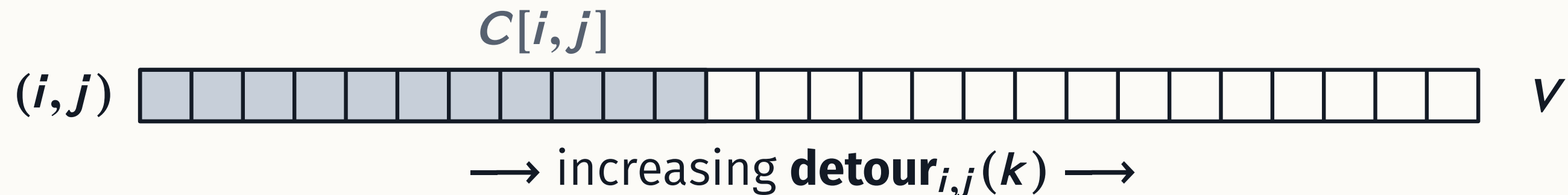
- ▶ Define the **detour of i and j via k** as $\mathbf{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$
- ▶ **Intuition:** Small detour $\implies k$ close to shortest path, good relaxation

Certificate:

A matrix $\mathbf{C}$ where $\mathbf{C}[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

Claim:

Can verify distance matrix $\mathbf{D}$ in time $\tilde{O}(n^{2.75})$, given correct certificate $\mathbf{C}$.



- ▶ **Step 1:** Verify that there is no decreasing relaxation in $\mathbf{C}[i,j]$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

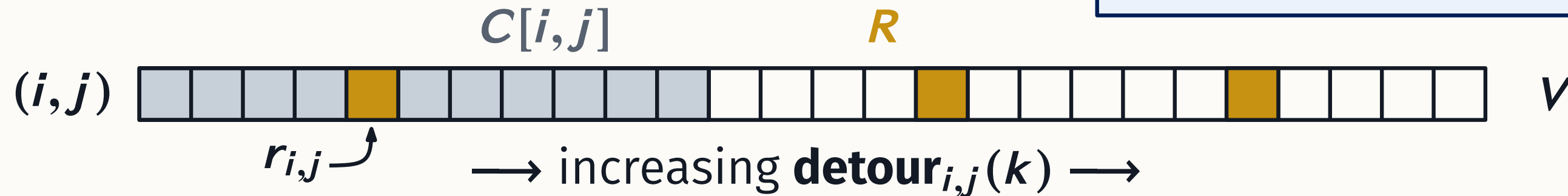


$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.



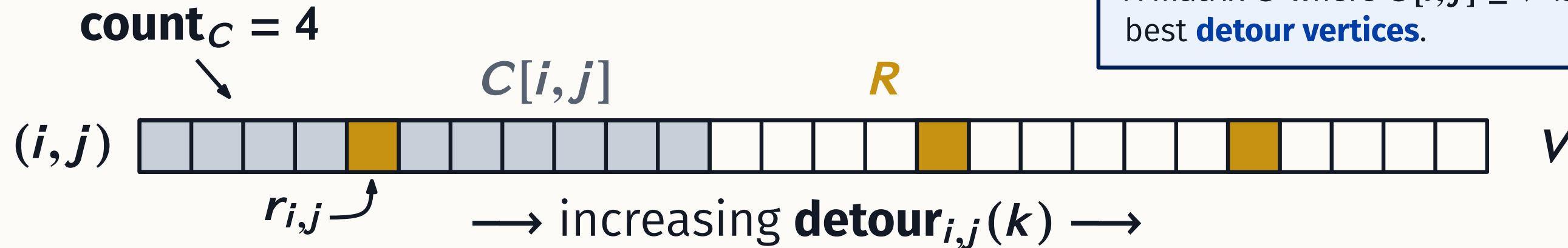
- Let $R \subseteq V$ be a **random hitting set** for all $C[i,j]$ with $r_{i,j} \in R \cap C[i,j]$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.



► Let $R \subseteq V$ be a **random hitting set for all $C[i,j]$** with $r_{i,j} \in R \cap C[i,j]$

► For every (i,j) , compute:

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

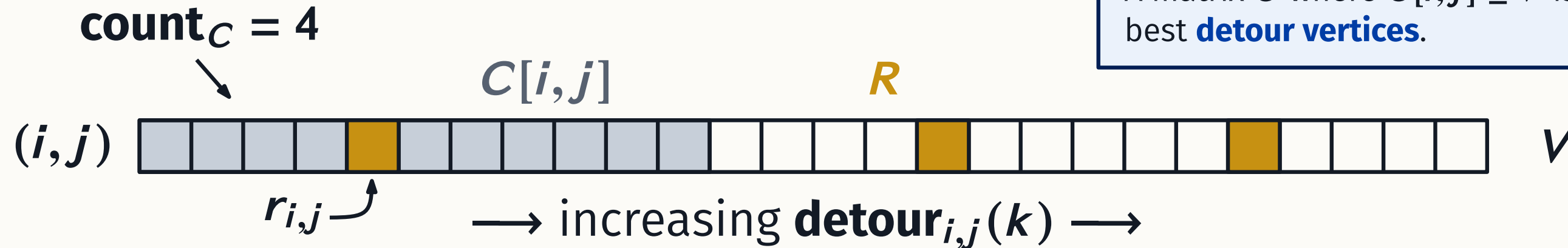
$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.



- ▶ Let $R \subseteq V$ be a **random hitting set for all $C[i,j]$** with $r_{i,j} \in R \cap C[i,j]$
- ▶ For every (i,j) , compute:

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

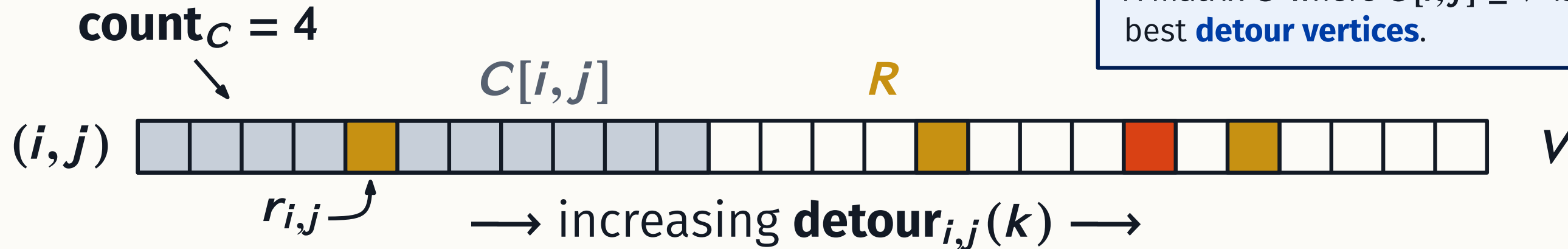
$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$
- ▶ If $\text{count}_C = \text{count}_V$ for all (i,j) , then no entry can be relaxed $\implies D$ correct

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

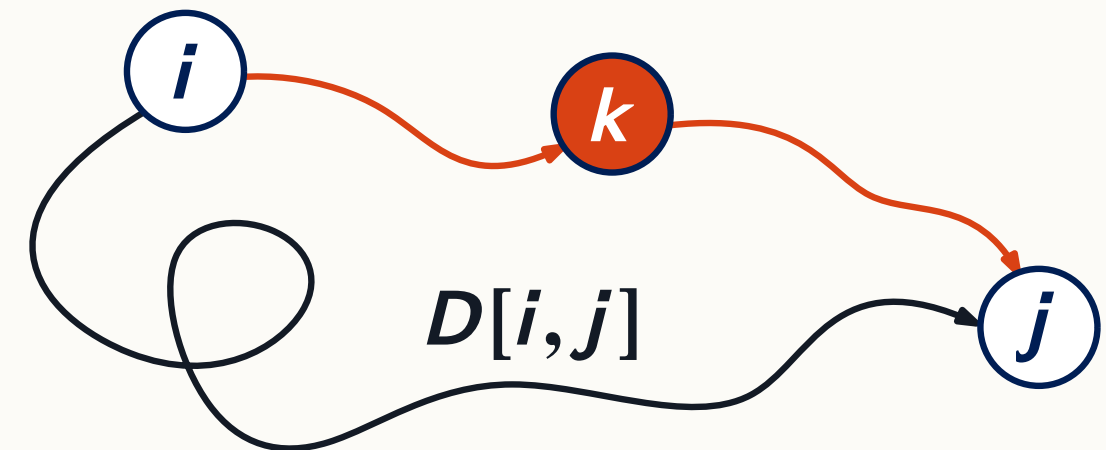


► Let $R \subseteq V$ be a **random hitting set for all $C[i,j]$** with $r_{i,j} \in R \cap C[i,j]$

► For every (i,j) , compute:

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$



► If $\text{count}_C = \text{count}_V$ for all (i,j) , then no entry can be relaxed $\Rightarrow D$ correct

► If $\text{count}_C < \text{count}_V$ for some (i,j) , there might be a **decreasing relaxation** somewhere

Sketch: APSP Verifier [CVWX]

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- Computing count_C is easy in time $|C| = n^{2.75}$

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing **count**_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute **count**_V?

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

count_C = $\#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

count_V = $\#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing **count**_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute **count**_V?
- ▶ For every $r \in R$ compute **count**_V **for all** (i,j) **with** $r_{i,j} = r$:

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

count_C = $\#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

count_V = $\#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing count_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute count_V ?
- ▶ For every $r \in R$ compute count_V **for all** (i,j) **with** $r_{i,j} = r$:

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing count_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute count_V ?
- ▶ For every $r \in R$ compute count_V **for all** (i,j) **with** $r_{i,j} = r$:

$$\begin{aligned} \text{count}_V &= \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\} \\ &= \#\{k \in V : D[i,k] + D[k,j] < D[i,r_{i,j}] + D[r_{i,j},j]\} \end{aligned}$$

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing **count**_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute **count**_V?
- ▶ For every $r \in R$ compute **count**_V **for all** (i,j) **with** $r_{i,j} = r$:

$$\begin{aligned} \text{count}_V &= \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\} \\ &= \#\{k \in V : D[i,k] + D[k,j] < D[i,r_{i,j}] + D[r_{i,j},j]\} \\ &= \#\{k \in V : D[i,k] - D[i,r_{i,j}] < D[r_{i,j},j] - D[k,j]\} \end{aligned}$$

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing **count**_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute **count**_V?
- ▶ For every $r \in R$ compute **count**_V **for all** (i,j) **with** $r_{i,j} = r$:

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\begin{aligned} \text{count}_V &= \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\} \\ &= \#\{k \in V : D[i,k] + D[k,j] < D[i,r_{i,j}] + D[r_{i,j},j]\} \\ &= \#\{k \in V : \underbrace{D[i,k] - D[i,r_{i,j}]}_{A[i,k]} < \underbrace{D[r_{i,j},j] - D[k,j]}_{B[k,j]}\} \\ &= \#\{k \in V : A[i,k] < B[k,j]\} \end{aligned}$$

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing count_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute count_V ?
- ▶ For every $r \in R$ compute count_V for all (i,j) with $r_{i,j} = r$:

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$= \#\{k \in V : D[i,k] + D[k,j] < D[i,r_{i,j}] + D[r_{i,j},j]\}$$

$$= \#\{k \in V : D[i,k] - D[i,r_{i,j}] < D[r_{i,j},j] - D[k,j]\}$$

$$= \#\{k \in V : \underbrace{D[i,k] - D[i,r_{i,j}]}_{A[i,k]} < \underbrace{D[r_{i,j},j] - D[k,j]}_{B[k,j]}\}$$

"Fredman's trick"

Dominance product

- ▶ After removing addition, can compute in time $\tilde{O}(n^{(\omega+3)/2}) = \tilde{O}(n^{2.5})$

[Matoušek 1991]

$$\text{detour}_{i,j}(k) := D[i,k] + D[k,j] - D[i,j]$$

Sketch: APSP Verifier [CVWX]

- ▶ Computing count_C is easy in time $|C| = n^{2.75}$
- ▶ How to compute count_V ?
- ▶ For every $r \in R$ compute count_V for all (i,j) with $r_{i,j} = r$:

Certificate:

A matrix C where $C[i,j] \subseteq V$ is the set of the $n^{0.75}$ best **detour vertices**.

R = hitting set of size $\tilde{O}(n^{0.25})$

$$\text{count}_C = \#\{k \in C[i,j] : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$\text{count}_V = \#\{k \in V : \text{detour}_{i,j}(k) < \text{detour}_{i,j}(r_{i,j})\}$$

$$= \#\{k \in V : D[i,k] + D[k,j] < D[i,r_{i,j}] + D[r_{i,j},j]\}$$

$$= \#\{k \in V : D[i,k] - D[i,r_{i,j}] < D[r_{i,j},j] - D[k,j]\}$$

$$= \#\{k \in V : \underbrace{D[i,k] - D[i,r_{i,j}]}_{A[i,k]} < \underbrace{D[r_{i,j},j] - D[k,j]}_{B[k,j]}\}$$

"Fredman's trick"

Dominance product

- ▶ After removing addition, can compute in time $\tilde{O}(n^{(\omega+3)/2}) = \tilde{O}(n^{2.5})$ [Matoušek 1991]
- ▶ Over all $r \in R$ it takes $|R| \cdot \tilde{O}(n^{2.5}) = \tilde{O}(n^{2.75})$

Turning the Verifier Into a Learning-Augmented Algorithm

Turning the Verifier Into an Algorithm with Predictions

Turning the Verifier Into an Algorithm with Predictions

- ▶ Use the certificate $\mathcal{C}$ as our prediction

Certificate:

A matrix $\mathcal{C}$ where $\mathcal{C}[i,j] \subseteq V$ contains the p best **detour vertices** for (i,j) .

Turning the Verifier Into an Algorithm with Predictions

- Use the certificate C as our prediction

Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the p best **detour vertices** for (i,j) .

Turning the Verifier Into an Algorithm with Predictions

- Use the certificate C as our prediction

3 steps:

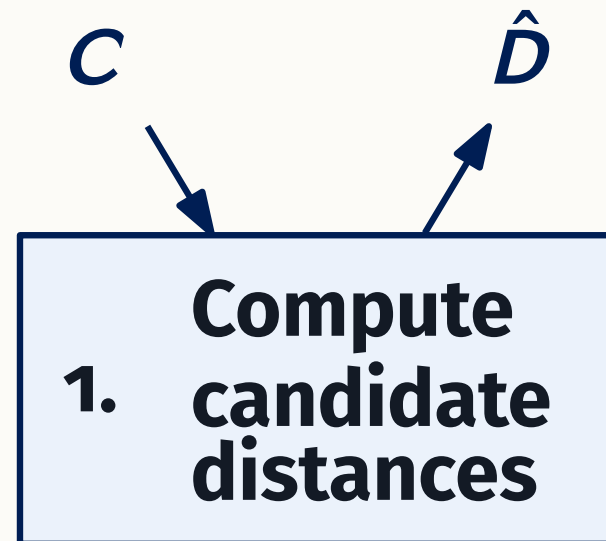
Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the p best **detour vertices** for (i,j) .

Turning the Verifier Into an Algorithm with Predictions

- Use the certificate C as our prediction

3 steps:



Prediction:

A matrix C where $C[i, j] \subseteq V$ **should contain** the p best **detour vertices** for (i, j) .

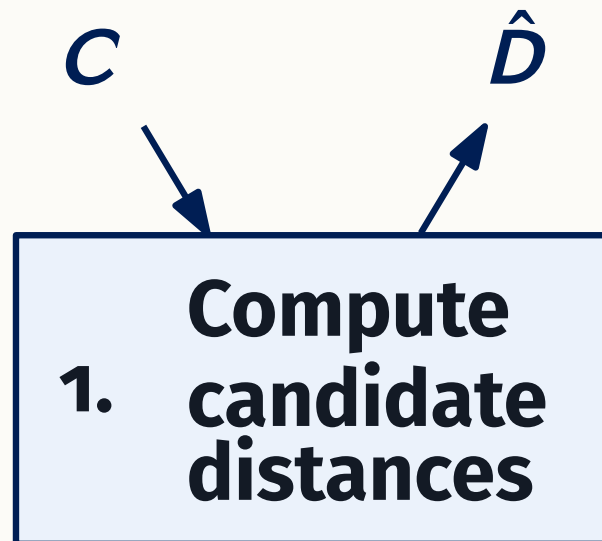
Turning the Verifier Into an Algorithm with Predictions

- Use the certificate C as our prediction

3 steps:

Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the p best **detour vertices** for (i,j) .



Lemma

(informal)

In time $\tilde{O}(|C|)$, we can compute “good candidate distances” $\hat{D}$ from C .

“perform **all and only** relaxations in C ” ← in a **good order**!

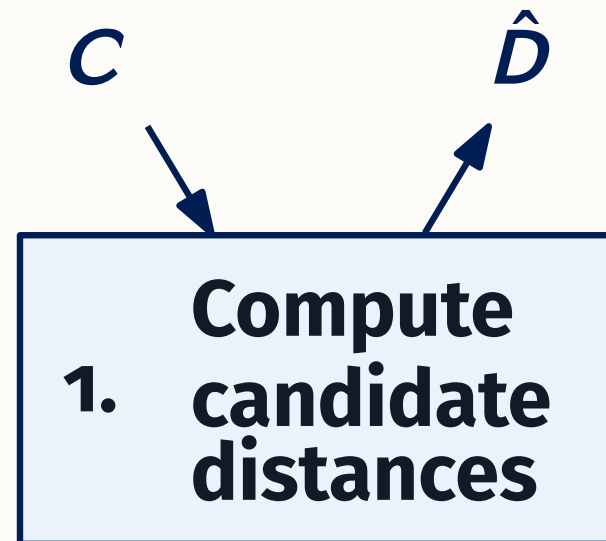
Turning the Verifier Into an Algorithm with Predictions

- Use the certificate C as our prediction

3 steps:

Prediction:

A matrix C where $C[i, j] \subseteq V$ **should contain** the p best **detour vertices** for (i, j) .



Lemma

(informal)

In time $\tilde{O}(|C|)$, we can compute “good candidate distances” $\hat{D}$ from C .

“perform **all and only** relaxations in C ” ← in a **good order**!

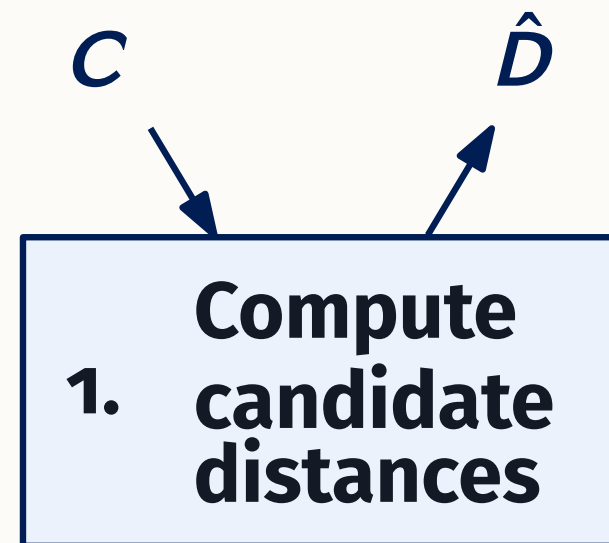
Turning the Verifier Into an Algorithm with Predictions

- Use the certificate C as our prediction

3 steps:

Prediction:

A matrix C where $C[i, j] \subseteq V$ **should contain** the p best **detour vertices** for (i, j) .



Lemma

(informal)

In time $\tilde{O}(|C|)$, we can compute “good candidate distances” $\hat{D}$ from C .

verifies each entry $\hat{D}[i, j]$ **locally**:

$$\hat{D}[i, j] \stackrel{?}{=} \min_{k \in V} \hat{D}[i, k] + \hat{D}[k, j]$$

“perform **all and only** relaxations in C ” ← in a **good order**!

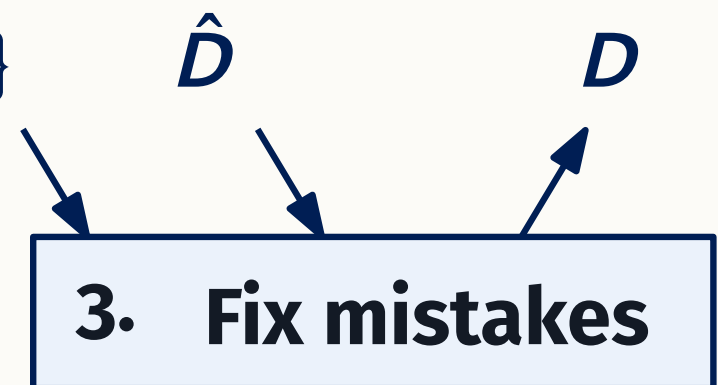
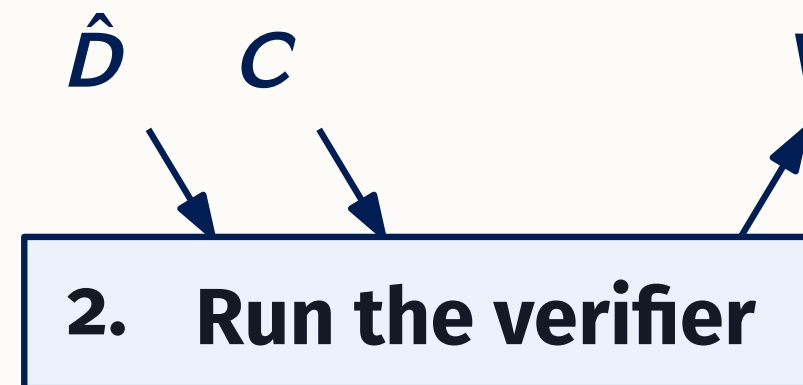
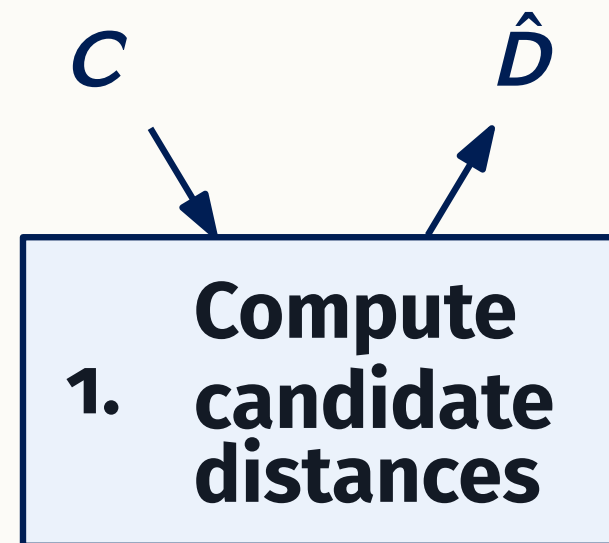
Turning the Verifier Into an Algorithm with Predictions

- Use the certificate C as our prediction

3 steps:

Prediction:

A matrix C where $C[i, j] \subseteq V$ **should contain** the p best **detour vertices** for (i, j) .



Lemma (informal)

In time $\tilde{O}(|C|)$, we can compute “good candidate distances” $\hat{D}$ from C .

verifies each entry $\hat{D}[i, j]$ **locally**:

$$\hat{D}[i, j] \stackrel{?}{=} \min_{k \in V} \hat{D}[i, k] + \hat{D}[k, j]$$

“perform **all and only** relaxations in C ” ← in a **good order**!

Error Measure

Theorem

Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

Prediction:

A matrix C where $C[i, j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Error Measure

Theorem

Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

Prediction:

A matrix C where $C[i, j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Error measure:

$$\eta \quad := \quad \#\{ (i, j) : \text{verifier failed} \} \quad + \quad \#\{ (i, j) : \hat{D}[i, j] \text{ incorrect} \}$$

Error Measure

Theorem

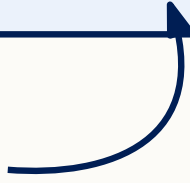
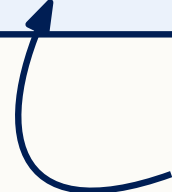
Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Verifier detects these  Might have incorrect $\hat{D}$  If not all best detours are in $C[i,j]$  Verifier might not detect 

Error Measure

Theorem

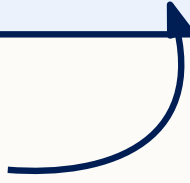
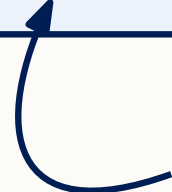
Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Verifier detects these   Might have incorrect $\hat{D}$  If not all best detours are in $C[i,j]$  Verifier might not detect

Want to perform **all $n \cdot \eta$ relaxations** only once!

Error Measure

Theorem

Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

Prediction:

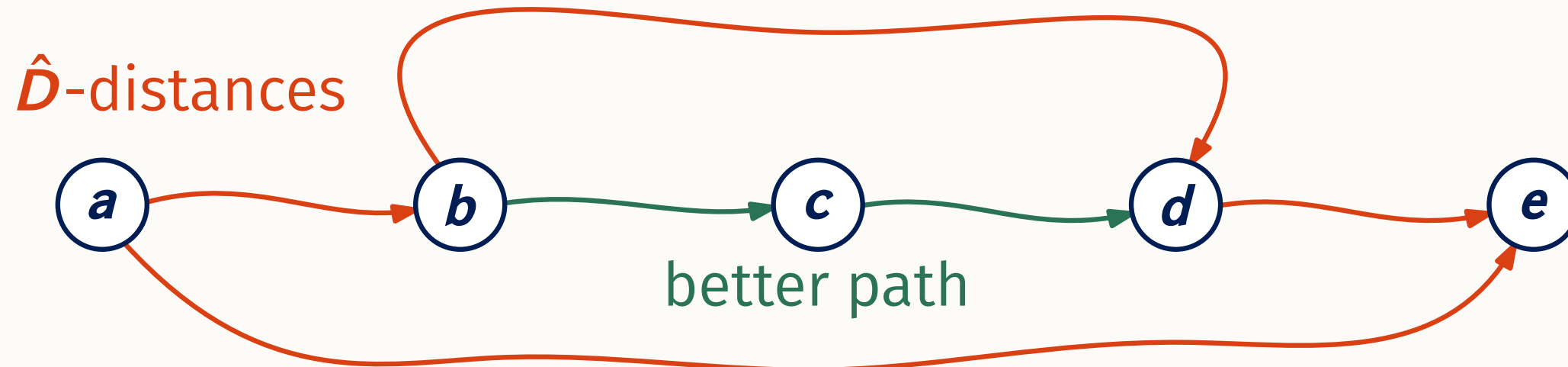
A matrix C where $C[i,j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Verifier detects these $\nearrow$ $\nearrow$ $\nearrow$ If not all best detours are in $C[i,j]$ $\nearrow$ Verifier might not detect
Might have incorrect $\hat{D}$

Want to perform **all $n \cdot \eta$ relaxations** only once!



Error Measure

Theorem

Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

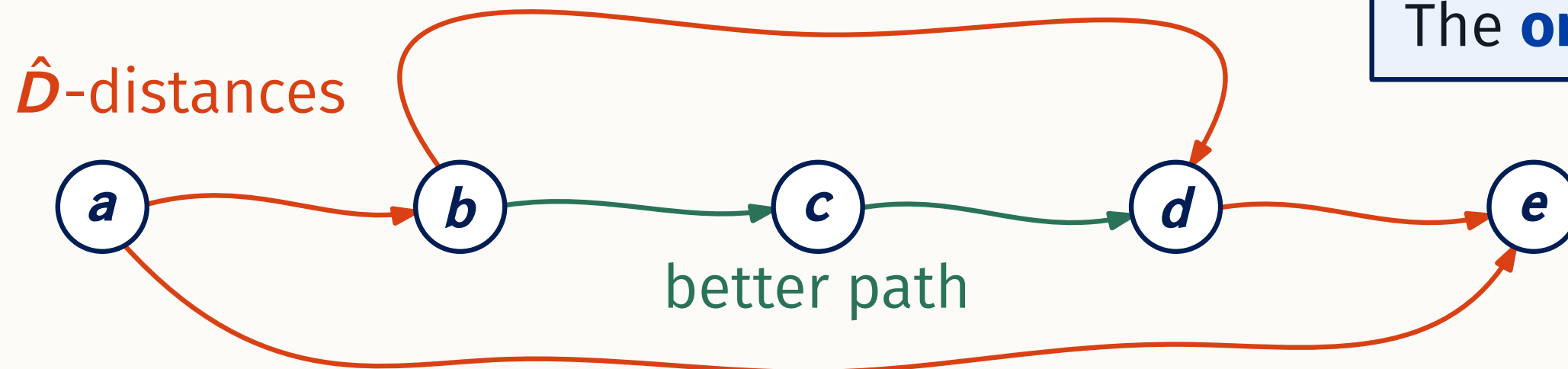
Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Verifier detects these $\nearrow$ $\nearrow$ $\nearrow$ If not all best detours are in $C[i,j]$ $\nearrow$ Verifier might not detect
Might have incorrect $\hat{D}$

Want to perform **all $n \cdot \eta$ relaxations** only once!

The **order of relaxations** matters!



Fixing: Algorithm Idea

Error measure:

$$\eta \quad := \quad \#\{ (i,j) : \text{verifier failed} \} \quad + \quad \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Fixing: Algorithm Idea

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

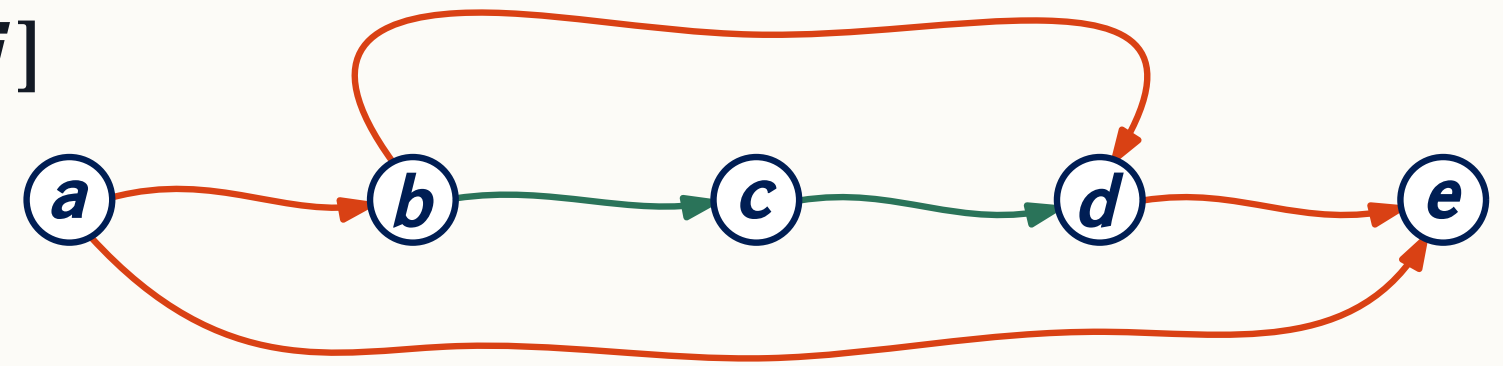
- Use **Johnson's trick** to assume w.l.o.g. that **weights are non-negative**

Fixing: Algorithm Idea

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

- ▶ Use **Johnson's trick** to assume w.l.o.g. that **weights are non-negative**
- ▶ Consider pairs in **increasing order** of $\hat{D}[i,j]$



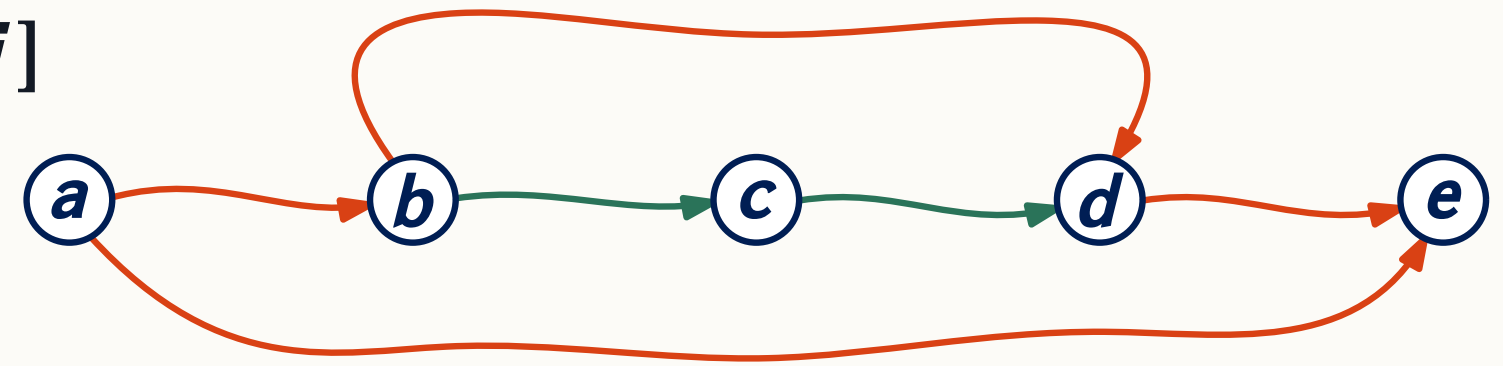
Fixing: Algorithm Idea

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

- ▶ Use **Johnson's trick** to assume w.l.o.g. that **weights are non-negative**
- ▶ Consider pairs in **increasing order** of $\hat{D}[i,j]$

Subpaths of shortest paths will be considered first!



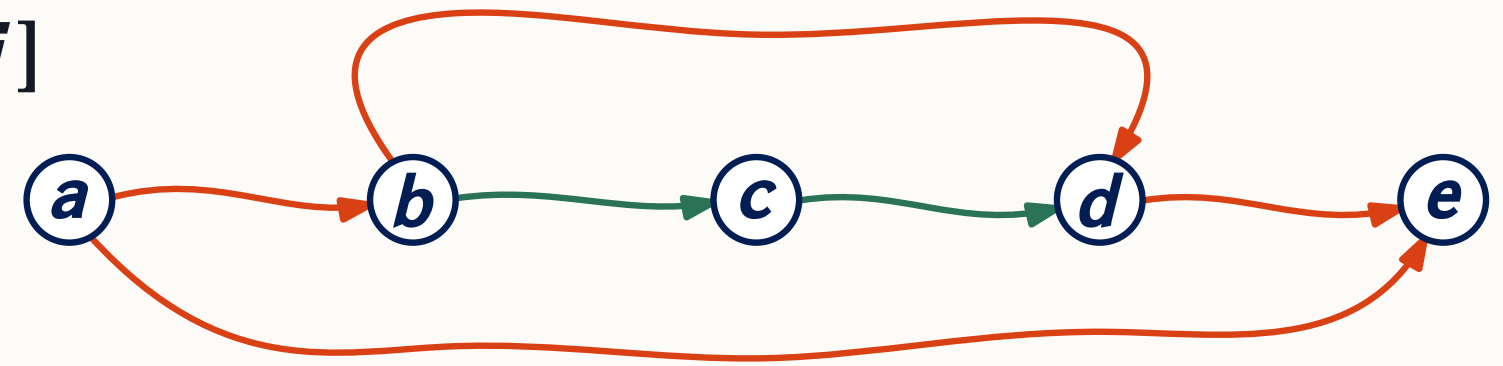
Fixing: Algorithm Idea

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

- ▶ Use **Johnson's trick** to assume w.l.o.g. that **weights are non-negative**
- ▶ Consider pairs in **increasing order** of $\hat{D}[i,j]$

Subpaths of shortest paths will be considered first!



- ▶ Do relaxations as **lazily** as possible

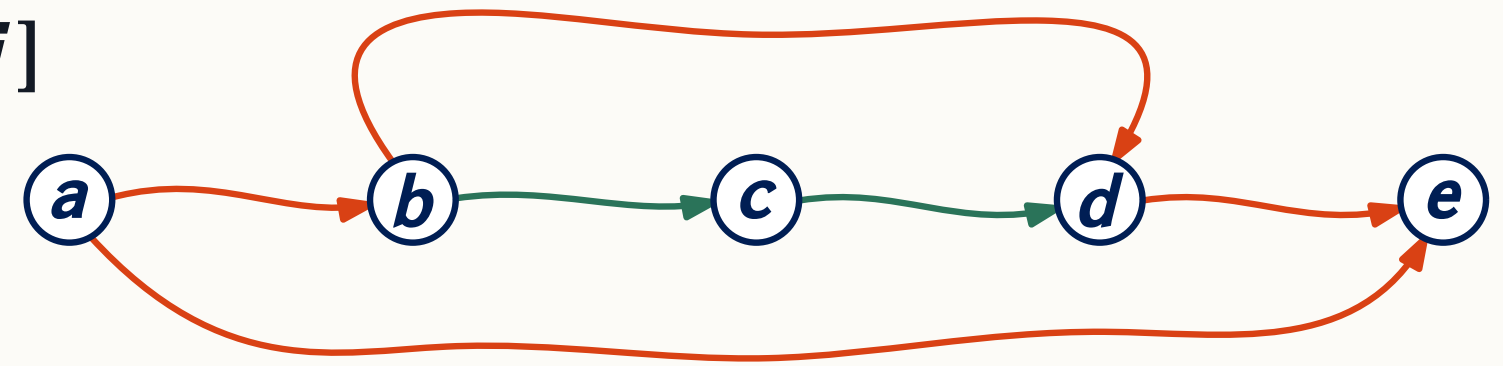
Fixing: Algorithm Idea

Error measure:

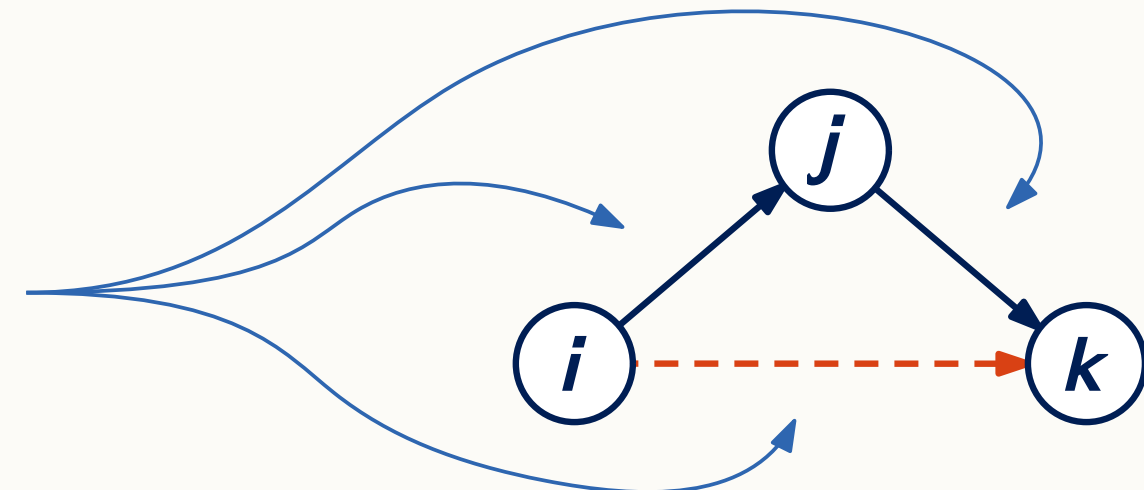
$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

- ▶ Use **Johnson's trick** to assume w.l.o.g. that **weights are non-negative**
- ▶ Consider pairs in **increasing order** of $\hat{D}[i,j]$

Subpaths of shortest paths will be considered first!



- ▶ Do relaxations as **lazily** as possible
- only if we know that one of these is counted in η



$$D[i,k] = \min \{ D[i,k], D[i,j] + D[j,k] \}$$

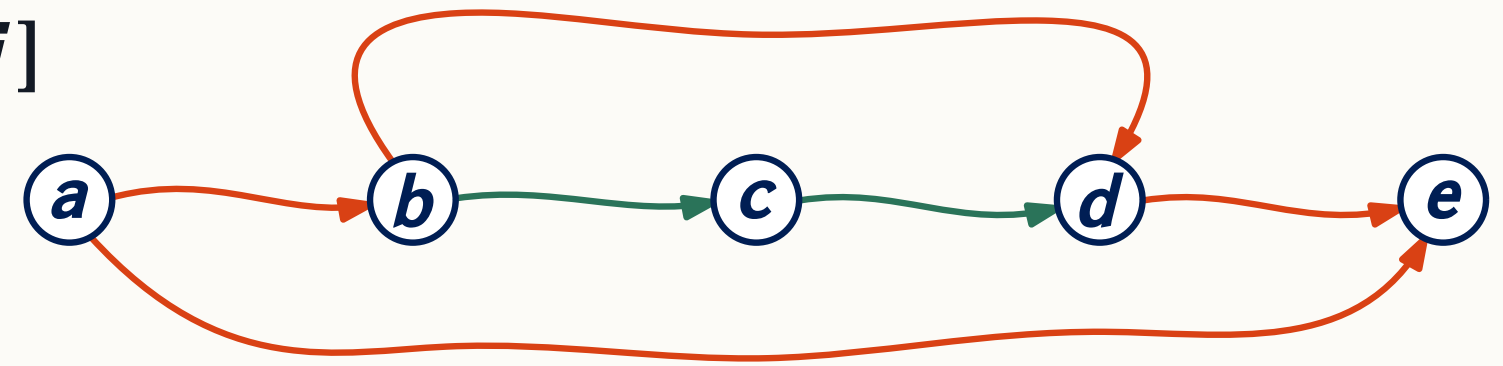
Fixing: Algorithm Idea

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

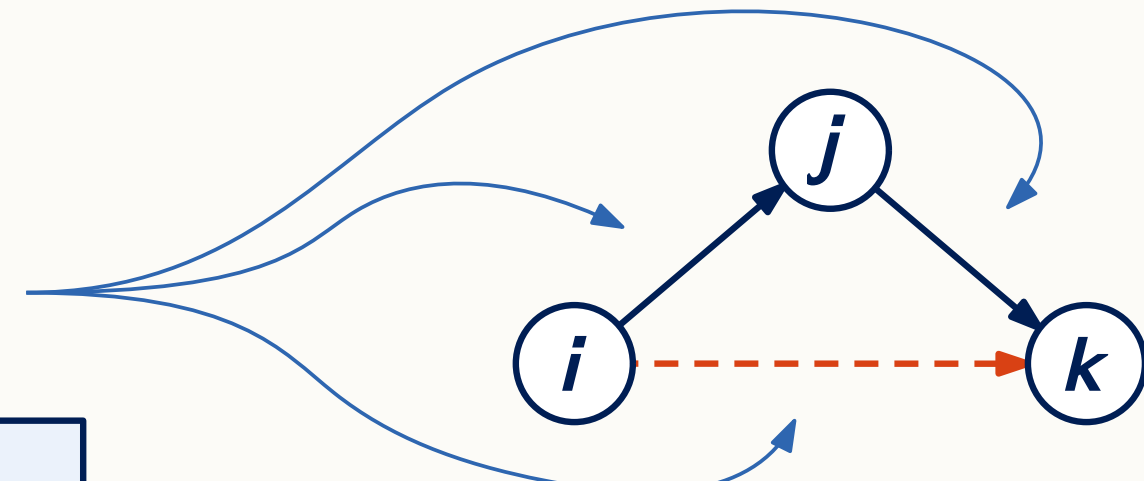
- ▶ Use **Johnson's trick** to assume w.l.o.g. that **weights are non-negative**
- ▶ Consider pairs in **increasing order** of $\hat{D}[i,j]$

Subpaths of shortest paths will be considered first!



- ▶ Do relaxations as **lazily** as possible

only if we know that one of these is counted in η



$$D[i,k] = \min \{ D[i,k], D[i,j] + D[j,k] \}$$

Total # relaxations:

$$\#\{ \text{total counted pairs} \} \cdot O(n) = O(\eta \cdot n)$$

Fixing: Full Algorithm

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

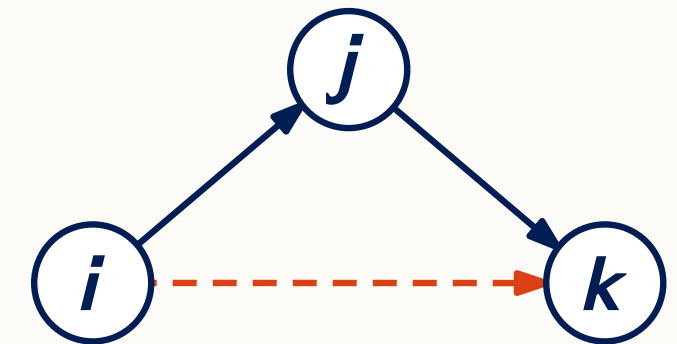
- ▶ Assume w.l.o.g. non-negative edge weights
- ▶ Keep a set of **marked pairs**, initially the set of unverified pairs
- ▶ Consider pairs (i,j) in **increasing order** of $\hat{D}[i,j]$:

- ▶ If (i,j) is marked:

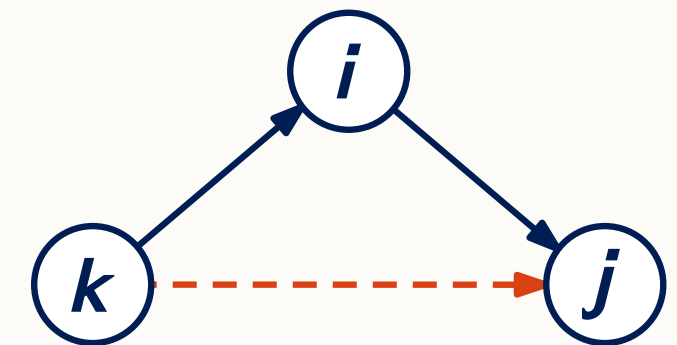
- ▶ Relax **every pair** with $\hat{D}[i,j]$
- ▶ If changed, mark

- ▶ If (i,j) is **not** marked:

- ▶ Relax **every marked pair** with $\hat{D}[i,j]$



$$D[i,k] = \min \{ D[i,k], D[i,j] + D[j,k] \}$$



$$D[k,j] = \min \{ D[k,j], D[k,i] + D[i,j] \}$$

Conclusion

Conclusion

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Prediction:

A matrix $\mathcal{C}$ where $\mathcal{C}[i,j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Conclusion

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Theorem

Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

1. Compute candidate distances

2. Run the verifier



3. Fix mistakes



Conclusion

Error measure:

$$\eta := \#\{ (i,j) : \text{verifier failed} \} + \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Prediction:

A matrix C where $C[i,j] \subseteq V$ **should contain** the $n^{0.75}$ best **detour vertices**.

Theorem

Given G and C , we can compute D in time $\tilde{O}(n\eta + n^{2.75})$.

1. Compute candidate distances

2. Run the verifier



3. Fix mistakes



Open Questions

Error measure:

$$\eta \quad := \quad \#\{ (i,j) : \text{verifier failed} \} \quad + \quad \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Error measure:

$$\eta \quad := \quad \#\{ (i,j) : \text{verifier failed} \} \quad + \quad \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

Open Questions

- ▶ **Remove $\#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$ from η ?**
 - ▶ Other term seems necessary to make verifier work...

Open Questions

Error measure:

$$\eta \quad := \quad \#\{ (i,j) : \text{verifier failed} \} \quad + \quad \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

- ▶ **Remove $\#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$ from η ?**
 - ▶ Other term seems necessary to make verifier work...
- ▶ **Learn the certificate?**
 - ▶ Exact ERM is NP-hard, approximation possible?

Open Questions

Error measure:

$$\eta \quad := \quad \#\{ (i,j) : \text{verifier failed} \} \quad + \quad \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

- ▶ **Remove $\#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$ from η ?**
 - ▶ Other term seems necessary to make verifier work...
- ▶ **Learn the certificate?**
 - ▶ Exact ERM is NP-hard, approximation possible?
- ▶ **Circumvent more lower bounds with predictions?**

Open Questions

Error measure:

$$\eta \quad := \quad \#\{ (i,j) : \text{verifier failed} \} \quad + \quad \#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$$

- ▶ **Remove $\#\{ (i,j) : \hat{D}[i,j] \text{ incorrect} \}$ from η ?**
 - ▶ Other term seems necessary to make verifier work...
- ▶ **Learn the certificate?**
 - ▶ Exact ERM is NP-hard, approximation possible?
- ▶ **Circumvent more lower bounds with predictions?**

Thank you!